

LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUT



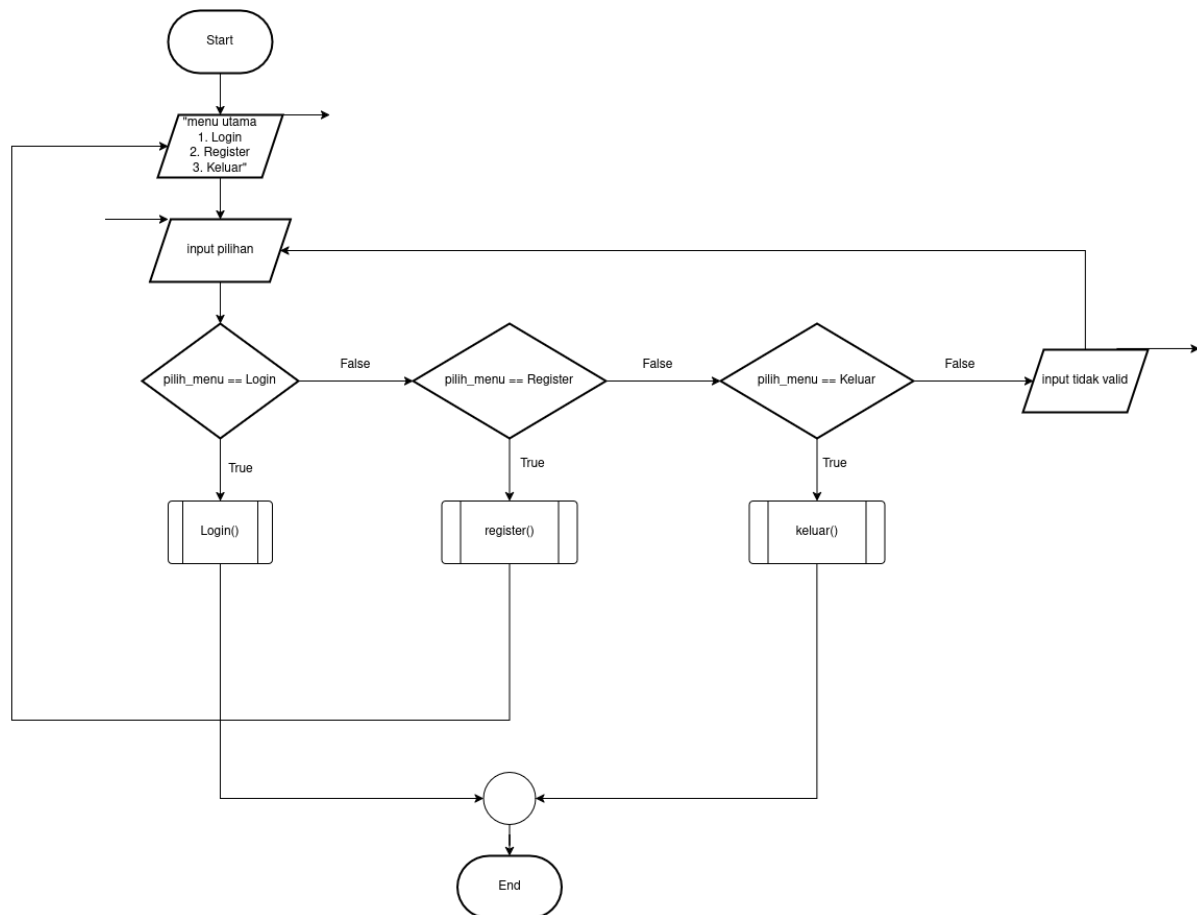
Disusun oleh:
Diftya Azzahra (2509106042)
Kelas (A2'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart

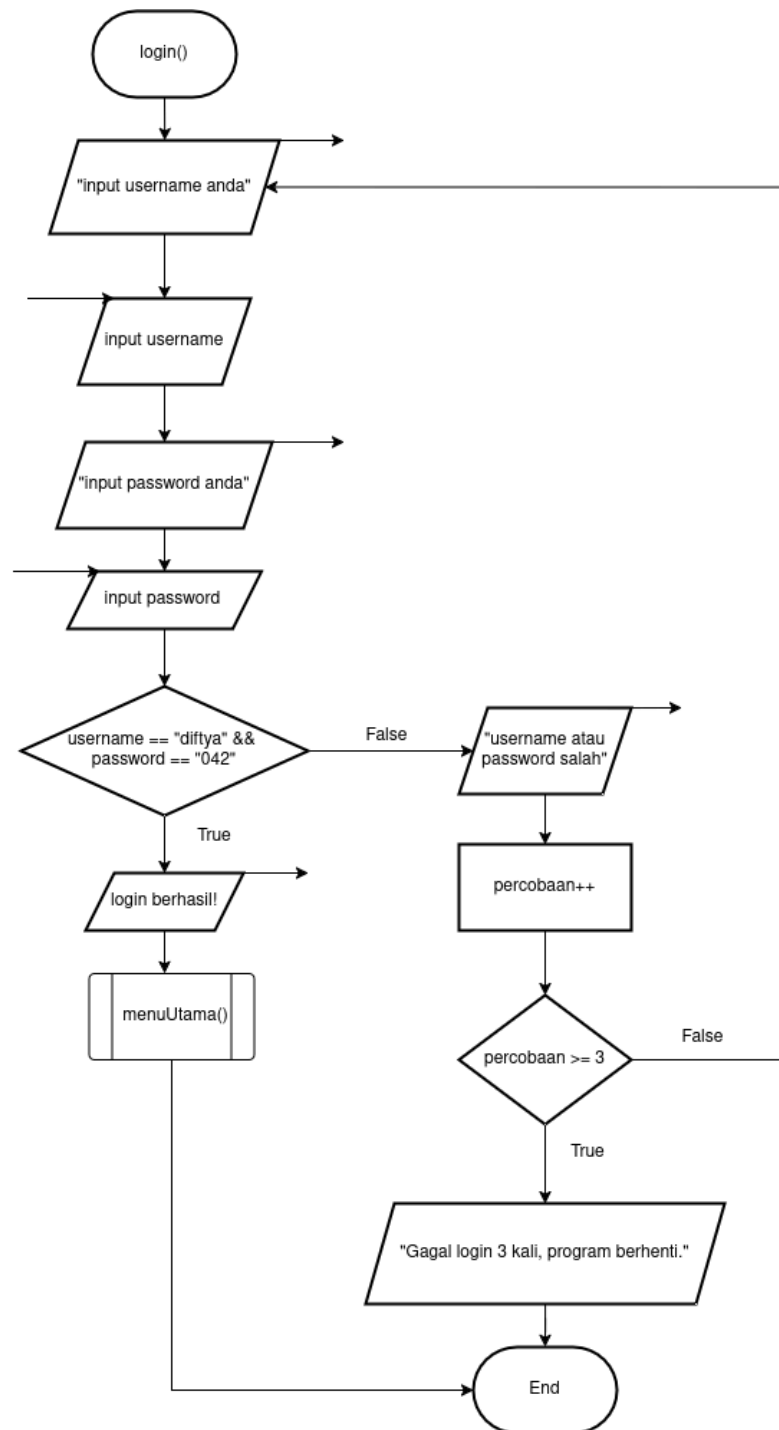
A. Admin

Pada bagian ini, flowchart akan dipisah menjadi bagian admin dan user. Penjelasan akan dimulai dari fitur admin terlebih dahulu



Gambar 1.1 Menu Awal

1. Proses dimulai dari Start, kemudian sistem menampilkan menu awal yang berisi tiga pilihan utama, yaitu login, register, dan keluar. Setelah menu ditampilkan, pengguna diminta untuk memasukkan pilihan sesuai dengan angka yang tersedia.
2. Setelah menginput, sistem melakukan percabangan berdasarkan pilihan pengguna. Jika pengguna memilih login, maka akan masuk ke dalam fungsi loginUser(). Jika pengguna memilih register, maka akan masuk ke dalam fungsi register() dan jika pengguna memilih keluar, maka akan masuk ke dalam fungsi keluar().
3. Jika pengguna menginput selain daripada pilihan yang diberikan, maka input akan dinyatakan tidak valid.

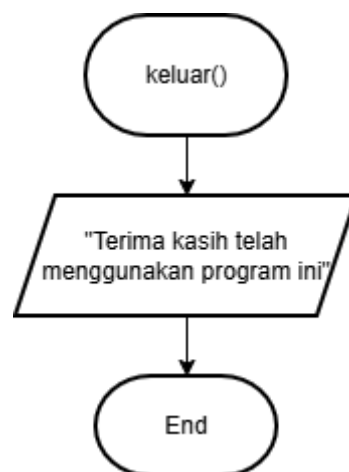


Gambar 1.2 Fungsi Login

4. Proses dimulai dari pemanggilan fungsi login(), di mana sistem akan menampilkan perintah kepada pengguna untuk memasukkan username. Setelah itu, pengguna menginput username serta password untuk lanjut ke tahap berikutnya.
5. Sistem kemudian melakukan pengecekan dengan membandingkan username dan password yang dimasukkan dengan data admin yang telah ditentukan, yaitu username

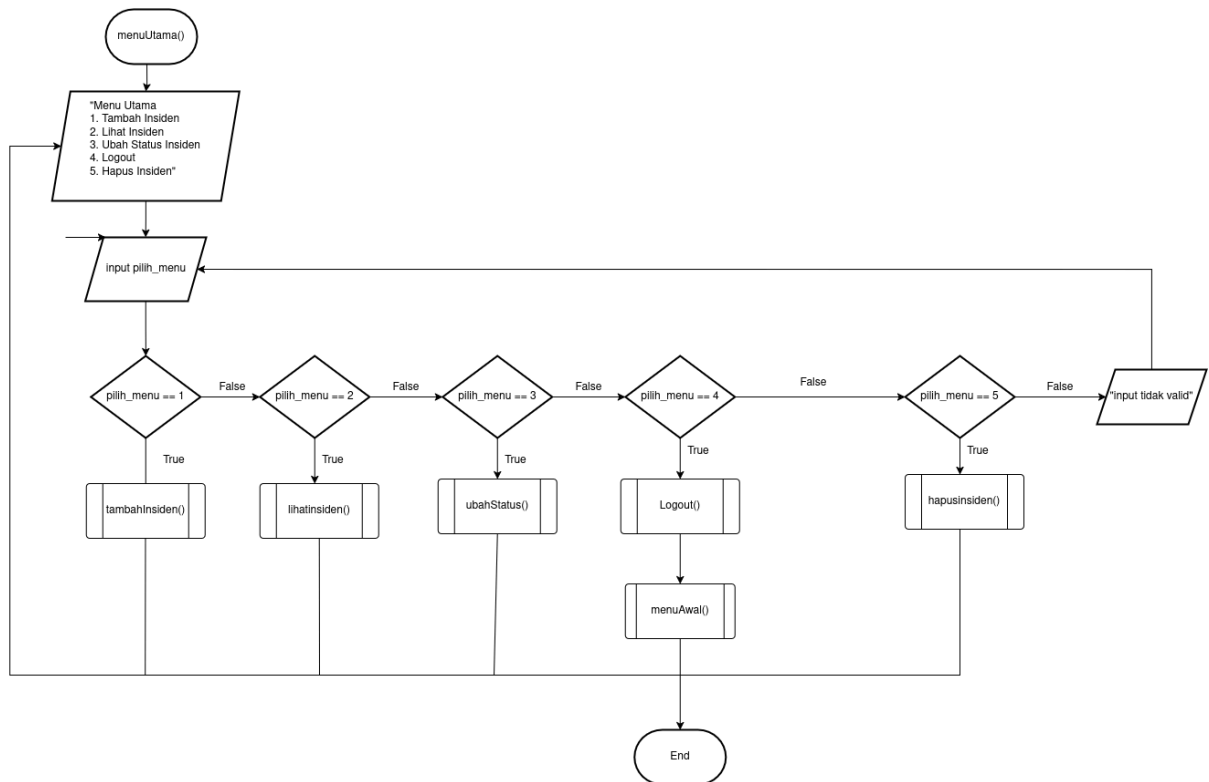
“difitya” dan password “042”. Jika kedua data tersebut sesuai, maka kondisi bernilai *true* dan sistem menampilkan pesan bahwa login berhasil, kemudian pengguna langsung diarahkan ke menu utama melalui pemanggilan fungsi `menuUtama()`.

6. Namun, jika username atau password tidak sesuai, maka kondisi bernilai *false* dan sistem akan menampilkan pesan bahwa username atau password salah. Setelah itu, variabel percobaan akan ditambahkan satu sebagai bentuk pencatatan jumlah kegagalan login yang telah dilakukan oleh pengguna.
7. Sistem kemudian akan melakukan pengecekan terhadap jumlah percobaan login. Jika jumlah percobaan masih kurang dari atau sama dengan tiga, maka pengguna akan dikembalikan ke proses input username dan password untuk mencoba kembali. Namun, jika jumlah percobaan telah melebihi batas yang ditentukan, yaitu lebih dari tiga kali, maka sistem akan menampilkan pesan bahwa login gagal sebanyak tiga kali dan program akan dihentikan.



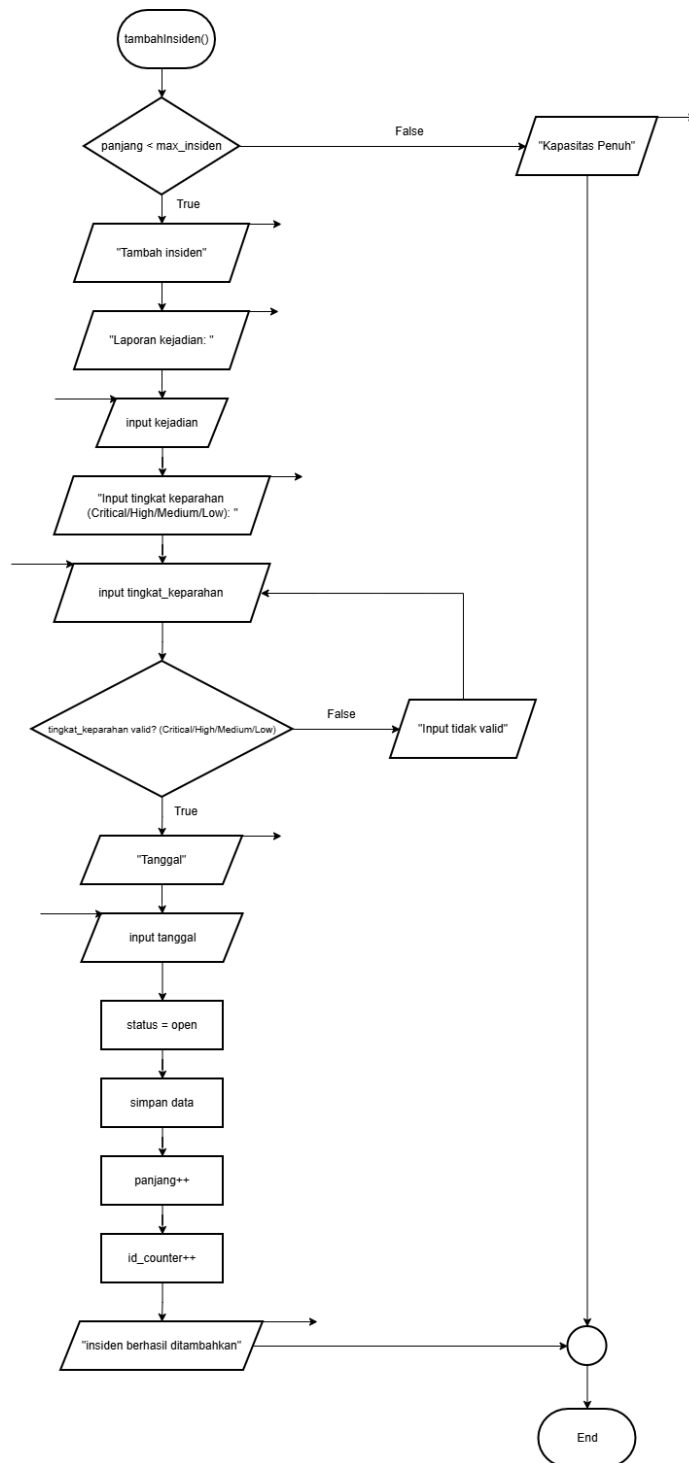
Gambar 1.3 Keluar Program

8. Fungsi ini akan dipanggil ketika pengguna memilih menu ketiga yaitu keluar program. Sistem akan menampilkan output berupa “Terima kasih telah menggunakan program ini” kemudian program pun berhenti.
9. END.



Gambar 1.4 Fungsi Menu Utama

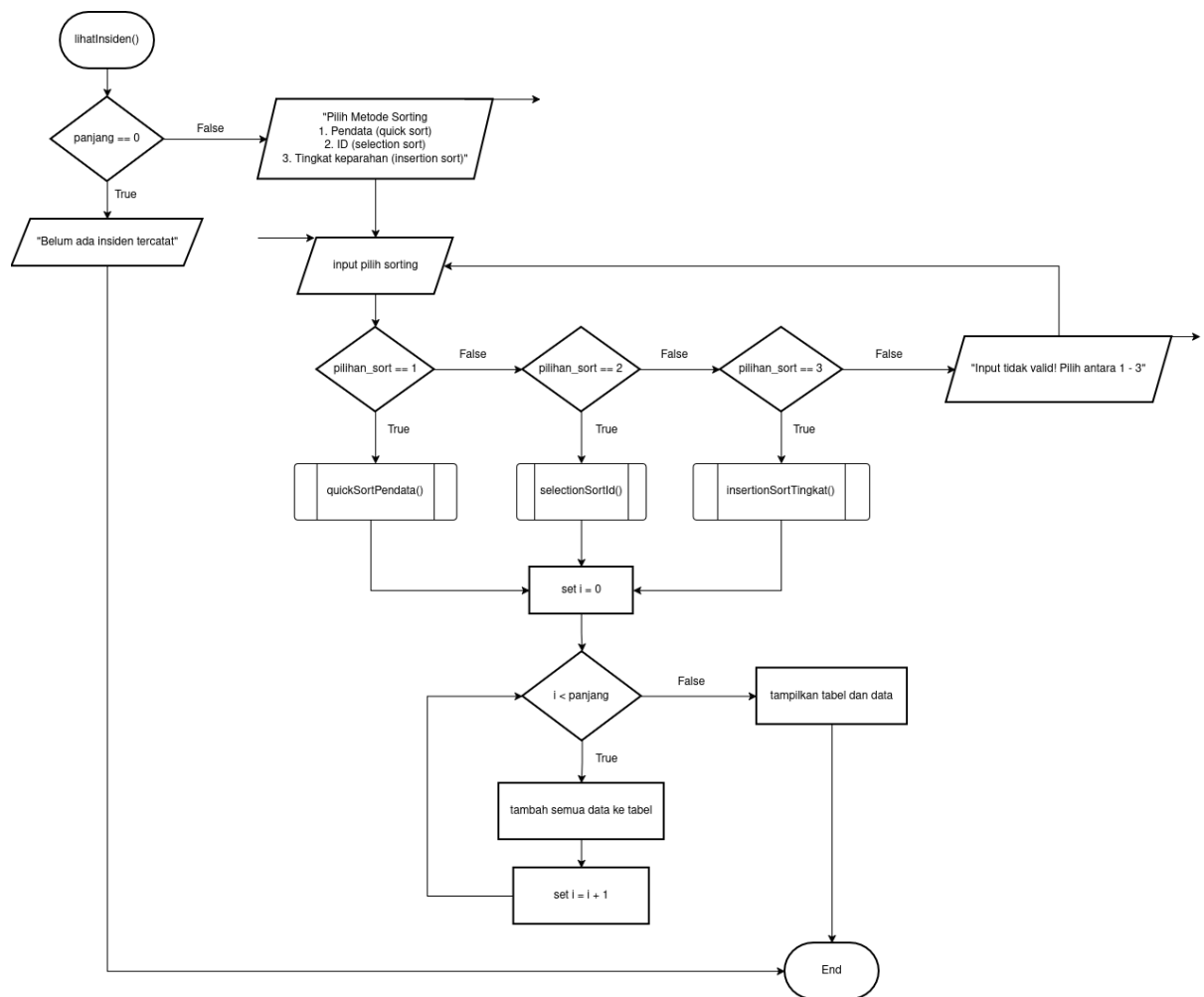
10. Proses dimulai dari pemanggilan fungsi menuUtama(), di mana sistem menampilkan daftar menu utama kepada pengguna yang telah berhasil login sebagai admin. Menu yang ditampilkan terdiri dari lima pilihan, yaitu tambah insiden, lihat insiden, ubah status insiden, logout, dan hapus insiden. Setelah itu, pengguna diminta untuk memasukkan pilihan menu yang diinginkan.
11. Setelah pengguna memasukkan pilihan, sistem akan melakukan pengecekan terhadap nilai input tersebut. Jika pengguna memilih menu dengan nilai 1, maka sistem akan menjalankan fungsi tambahInsiden(). Jika pengguna memilih menu dengan nilai 2, maka sistem akan menjalankan fungsi lihatInsiden(). Jika pengguna memilih menu dengan nilai 3, maka sistem akan menjalankan fungsi ubahStatus(). Jika pengguna memilih menu dengan nilai 4, maka sistem akan menjalankan fungsi logout() dan jika pengguna memilih menu dengan nilai 5, maka sistem akan menjalankan fungsi hapusInsiden().
12. Namun, jika input yang diberikan tidak sesuai dengan pilihan yang tersedia, maka sistem akan menganggapnya sebagai input tidak valid dan tidak menjalankan fungsi apa pun, sehingga sistem akan kembali menunggu input yang benar sesuai dengan implementasi program.



Gambar 1.5 Fungsi Tambah Insiden

13. Proses dimulai dari pemanggilan fungsi `tambahInsiden()`, di mana sistem pertama-tama melakukan pengecekan terhadap kapasitas penyimpanan data insiden dengan membandingkan nilai `panjang` dengan `max_insiden`. Jika kapasitas sudah penuh, maka kondisi bernilai `false` dan sistem akan langsung menampilkan pesan bahwa kapasitas penuh, kemudian proses berakhir tanpa menambahkan data baru.

14. Jika kapasitas masih tersedia, maka kondisi bernilai *true* dan sistem melanjutkan proses dengan menampilkan perintah untuk menambahkan insiden. Selanjutnya pengguna diminta untuk memasukkan laporan kejadian.
15. Setelah laporan kejadian diisi, sistem meminta pengguna untuk memasukkan tingkat keparahan insiden dengan pilihan yang telah ditentukan, yaitu *Critical*, *High*, *Medium*, atau *Low*.
16. Jika tingkat keparahan telah diinput, maka sistem akan melanjutkan dengan meminta input tanggal kejadian.
17. Setelah seluruh data yang dibutuhkan telah lengkap dan valid, sistem akan menetapkan status awal insiden sebagai “*open*”, kemudian seluruh data seperti ID, kejadian, tingkat keparahan, status, tanggal, dan pendata akan disimpan ke dalam struktur data array `dataInsiden()`.
18. Setelah data berhasil disimpan, sistem akan menambahkan nilai panjang sebagai penanda jumlah data insiden yang tersimpan serta menaikkan `id_counter` untuk memastikan setiap insiden memiliki ID yang unik. Terakhir, sistem menampilkan pesan bahwa insiden berhasil ditambahkan.

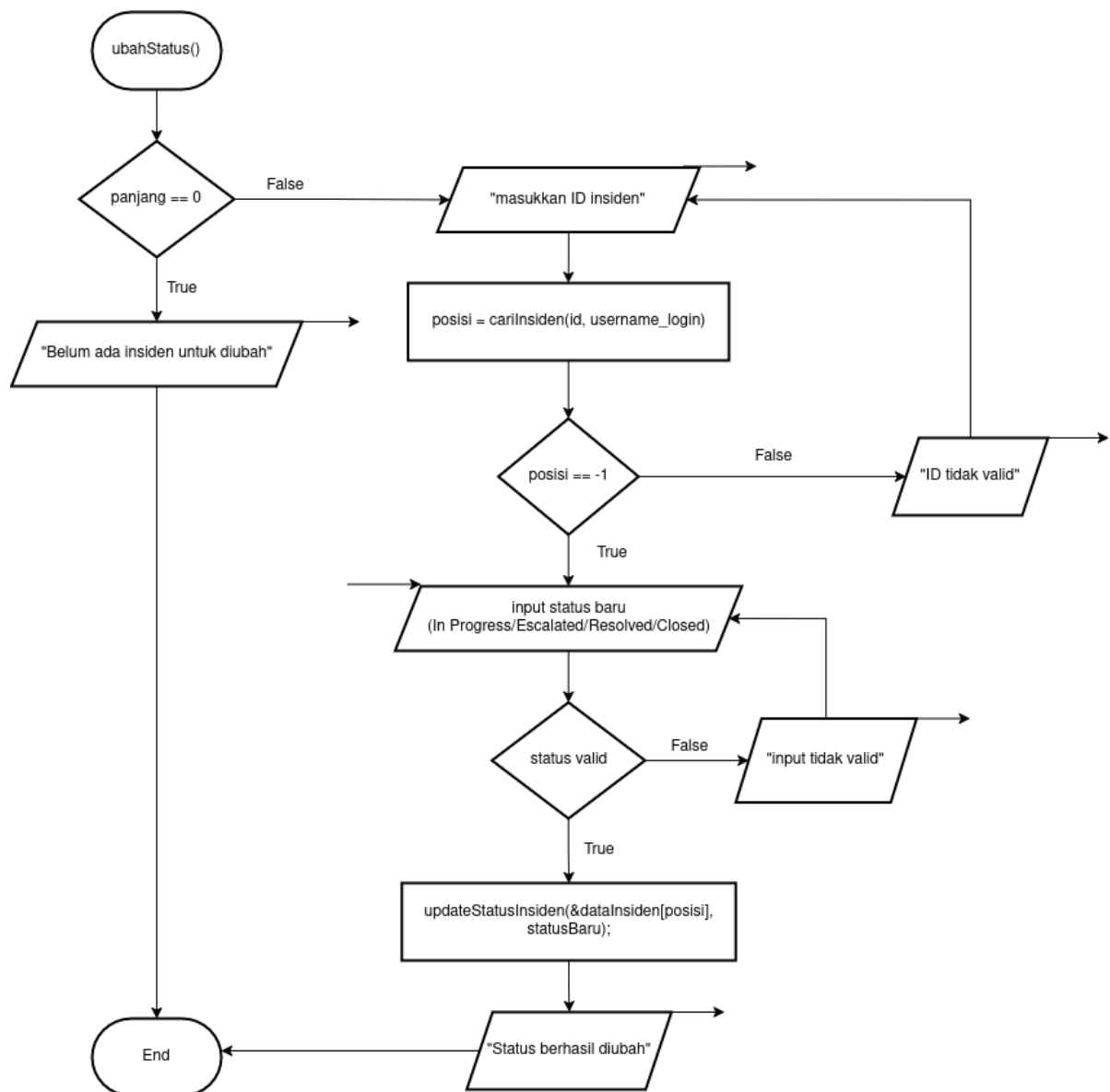


Gambar 1.6 Fungsi Lihat Insiden

19. Proses dimulai dari pemanggilan fungsi `lihatInsiden()` ketika admin memilih menu untuk melihat data insiden. Pada tahap awal, sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan memeriksa kondisi `panjang == 0`. Jika kondisi bernilai `true`, berarti belum ada data insiden yang tersimpan di dalam sistem, sehingga sistem menampilkan pesan “Belum ada insiden tercatat”, lalu proses berakhir.
20. Jika kondisi `panjang == 0` bernilai `false`, maka sistem melanjutkan proses dengan menampilkan pilihan metode sorting kepada admin. Pilihan yang tersedia adalah: (1) Pendata (Quick Sort), (2) ID (Selection Sort), (3) Tingkat Keparahan (Insertion Sort). Setelah pilihan metode sorting ditampilkan, sistem meminta admin untuk melakukan input pilih sorting.
21. Berikutnya, sistem memeriksa nilai input yang dimasukkan. Jika `pilihan_sort == 1`, maka sistem menjalankan fungsi `quickSortPendata()` untuk mengurutkan data berdasarkan nama pendata. Jika `pilihan_sort == 2`, maka sistem menjalankan fungsi

selectionSortId() untuk mengurutkan data dengan ascending berdasarkan ID. Jika pilihan_sort == 3, maka sistem menjalankan fungsi insertionSortTingkat() untuk mengurutkan data berdasarkan tingkat keparahan. Namun, jika input tidak sesuai dengan ketiga pilihan tersebut, maka sistem menampilkan pesan “Input tidak valid! Pilih antara 1 - 3” dan admin diminta untuk melakukan input ulang.

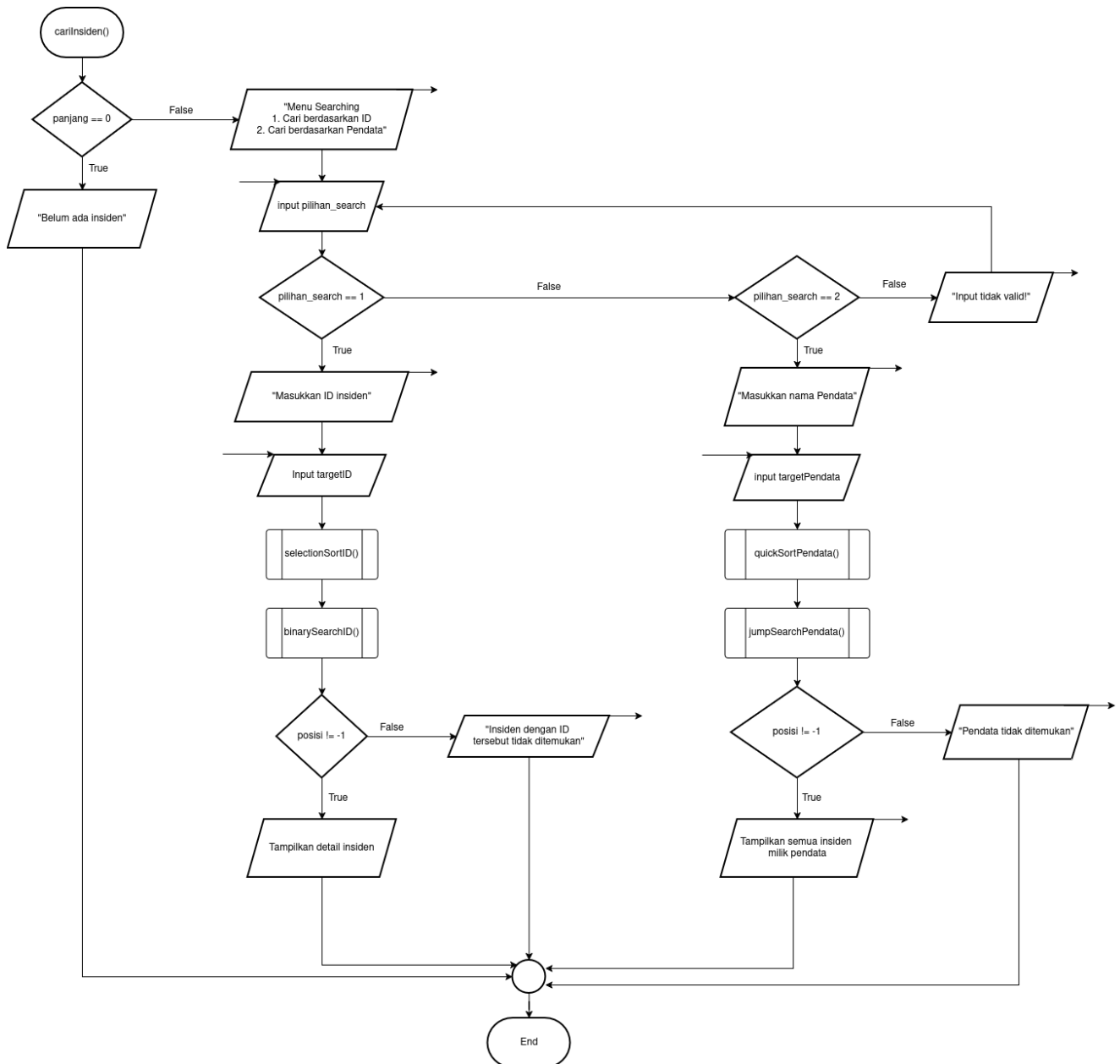
22. Setelah salah satu proses sorting selesai dijalankan, sistem menginisialisasi variabel $i = 0$ untuk memulai penelusuran seluruh data insiden. Selanjutnya, sistem memeriksa kondisi $i < \text{panjang}$. Kondisi ini digunakan untuk menentukan apakah masih ada data insiden yang perlu diproses. Jika kondisi bernilai true, maka sistem akan menambahkan seluruh data ke dalam tabel. Hal ini sesuai dengan hak akses admin yang dapat melihat semua data tanpa perlu melakukan pengecekan apakah data tersebut milik user tertentu atau bukan.
23. Setelah satu data ditambahkan ke tabel, sistem menaikkan nilai i dengan perintah $i = i + 1$, lalu kembali ke pengecekan $i < \text{panjang}$. Proses ini berlangsung berulang sampai seluruh data insiden selesai dimasukkan ke dalam tabel.
24. Jika kondisi $i < \text{panjang}$ bernilai false, berarti seluruh data telah selesai diproses. Setelah itu, sistem menampilkan tabel dan data hasil pengurutan kepada admin. Karena flowchart ini menggambarkan alur dari fungsi yang dipanggil di dalam menu utama, maka setelah tabel ditampilkan alur kembali ke menuUtama(), kemudian proses berakhir pada End.
25. Secara keseluruhan, flowchart lihatInsiden() versi admin menggambarkan bahwa admin dapat melihat seluruh data insiden yang ada di sistem. Sistem hanya perlu memeriksa apakah data tersedia atau tidak, lalu melakukan sorting sesuai pilihan admin, memasukkan semua data ke dalam tabel, dan akhirnya menampilkan tabel tersebut ke layar. Berbeda dengan versi user, pada flowchart admin tidak terdapat proses pengecekan $\text{pendata} == \text{username_login}$, karena admin memang memiliki akses penuh untuk melihat seluruh data insiden.



Gambar 1.7 Fungsi Ubah Status

26. Proses dimulai dari pemanggilan menu ubah status insiden pada fungsi menuUtama(), tepatnya pada pilihan menu ke-3.
27. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel panjang. Jika panjang == 0, maka kondisi bernilai true dan sistem menampilkan pesan “Belum ada Insiden untuk diubah”, kemudian proses langsung berakhir tanpa ada perubahan data.
28. Jika terdapat data insiden (panjang != 0), maka kondisi bernilai false dan sistem melanjutkan proses dengan meminta pengguna untuk memasukkan ID insiden.

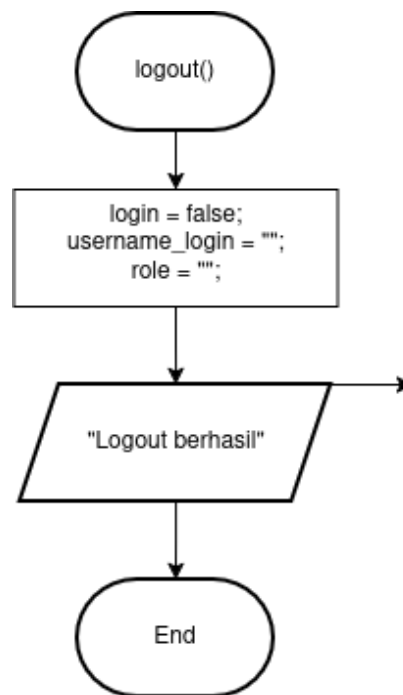
29. Setelah pengguna memasukkan ID, sistem akan mencari posisi data insiden dalam array menggunakan fungsi cariInsiden, baik untuk admin maupun user sesuai hak aksesnya.
30. Hasil pencarian disimpan dalam variabel posisi. Jika posisi == -1, maka ID tidak ditemukan atau tidak valid, sehingga sistem menampilkan pesan “ID tidak valid!” dan pengguna diminta untuk menginput ulang ID hingga valid.
31. Jika ID valid (posisi != -1), maka sistem melanjutkan dengan meminta pengguna memasukkan status baru untuk insiden tersebut.
32. Pengguna diminta menginput status dengan pilihan yang telah ditentukan, yaitu *In Progress*, *Escalated*, *Resolved*, atau *Closed*.
33. Sistem kemudian melakukan validasi terhadap input status. Jika status yang dimasukkan tidak sesuai dengan pilihan yang tersedia, maka sistem menampilkan pesan “Input tidak valid!” dan pengguna diminta menginput ulang hingga status valid.
34. Jika status yang dimasukkan valid, maka sistem akan melakukan proses pembaruan data dengan mengubah nilai.
35. Setelah proses update berhasil dilakukan, sistem menampilkan pesan “Status berhasil diubah” sebagai konfirmasi bahwa perubahan telah berhasil.
36. Proses kemudian berakhir dan pengguna dikembalikan ke menu utama untuk melanjutkan aktivitas lainnya.



Gambar 1.8 Fungsi Cari Insiden

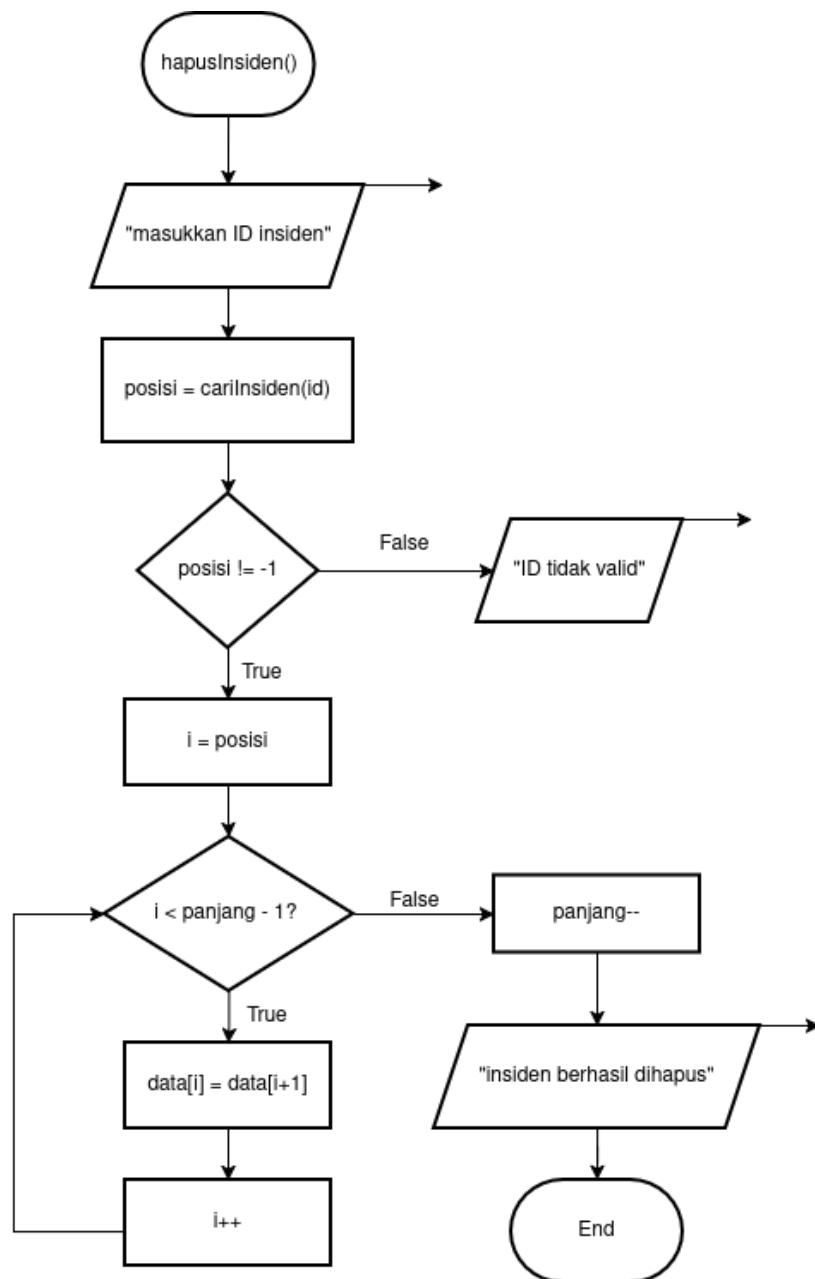
37. Pada flowchart cariInsiden(), proses dimulai ketika sistem memeriksa apakah jumlah data insiden sama dengan nol ($\text{panjang} == 0$). Jika tidak ada data insiden yang tersimpan, maka sistem akan menampilkan pesan “Belum ada insiden” dan proses pencarian selesai. Namun, jika data insiden tersedia, sistem akan menampilkan menu searching yang berisi dua pilihan, yaitu cari berdasarkan ID dan cari berdasarkan pendata.

38. Setelah menu pencarian ditampilkan, admin diminta memasukkan pilihan pencarian melalui variabel pilihan_search. Pada proses ini, admin dapat memilih nilai 1 untuk pencarian berdasarkan ID atau nilai 2 untuk pencarian berdasarkan pendata.
39. Jika admin memilih pencarian berdasarkan ID, sistem akan meminta admin memasukkan ID insiden yang ingin dicari. Nilai tersebut kemudian disimpan ke dalam variabel targetID. Setelah menginputkan ID, sistem akan mengurutkan data insiden berdasarkan ID menggunakan fungsi selectionSortId(). Proses pengurutan ini dilakukan agar pencarian dapat dilanjutkan dengan metode Binary Search. Setelah data terurut, sistem menjalankan fungsi binarySearchID() untuk mencari posisi data berdasarkan ID yang telah dimasukkan. Hasil pencarian tersebut kemudian disimpan ke dalam variabel posisi.
40. Selanjutnya, sistem memeriksa hasil pencarian ID. Jika nilai posisi == -1, berarti data insiden dengan ID tersebut tidak ditemukan, sehingga sistem menampilkan pesan “Insiden dengan ID tersebut tidak ditemukan.” Jika nilai posisi tidak sama dengan -1, berarti data insiden berhasil ditemukan, sehingga sistem menampilkan pesan “Insiden Ditemukan!” beserta detail insiden.
41. Apabila admin memilih pencarian berdasarkan pendata, sistem akan meminta admin memasukkan nama pendata yang ingin dicari. Input tersebut disimpan ke dalam variabel targetPendata. Setelah itu, sistem mengurutkan data insiden berdasarkan nama pendata menggunakan fungsi quickSortPendata(). Setelah data terurut, sistem melanjutkan pencarian menggunakan fungsi jumpSearchPendata(). Hasil pencarian nama pendata juga kemudian disimpan ke dalam variabel posisi.
42. Kemudian, sistem memeriksa apakah nama pendata berhasil ditemukan. Jika nilai posisi sama dengan -1, maka sistem menampilkan pesan “Pendata tidak ditemukan.” Namun, jika nilai posisi tidak sama dengan -1, maka sistem melanjutkan proses untuk menampilkan seluruh data insiden yang dimiliki oleh pendata tersebut. Sistem terlebih dahulu menelusuri ke posisi awal data dengan nama pendata yang sama, lalu menampilkan semua insiden milik pendata tersebut ke dalam bentuk tabel. Setelah itu, sistem menampilkan daftar insiden berdasarkan nama pendata yang dicari.



Gambar 1.9 Fungsi Logout

43. Proses dimulai dari pemanggilan fungsi logout() pada fungsi menuUtama(), tepatnya pada pilihan menu ke-4.
44. Sistem kemudian menjalankan proses penghapusan sesi login dengan mengubah nilai variabel login menjadi false, yang menandakan bahwa pengguna sudah tidak dalam keadaan login.
45. Selanjutnya, sistem mengosongkan data pengguna yang sedang aktif dengan mengatur nilai username_login menjadi string kosong ("").
46. Sistem juga mengosongkan informasi peran (role) pengguna dengan mengubah nilai variabel role menjadi string kosong ("").
47. Setelah seluruh data sesi dihapus, sistem menampilkan pesan “Logout berhasil” sebagai konfirmasi bahwa proses logout telah dilakukan dengan sukses.
48. Proses kemudian berakhir (End), dan pengguna akan dikembalikan ke kondisi awal sistem sebelum login.



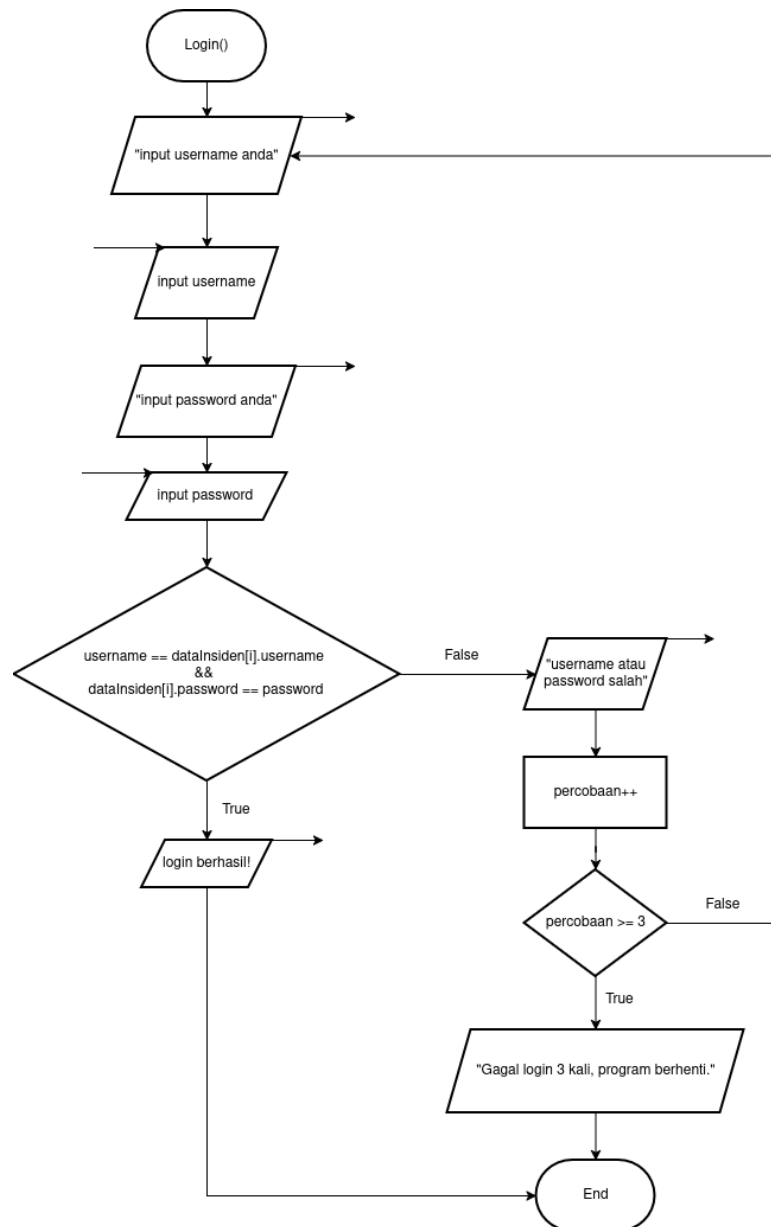
Gambar 1.10 Fungsi HapusInsiden

49. Proses dimulai dari pemanggilan fungsi hapusInsiden() yang terjadi ketika pengguna (admin) memilih menu untuk menghapus data insiden.
50. Sistem kemudian meminta admin untuk memasukkan ID insiden yang ingin dihapus dengan menampilkan pesan “masukkan ID insiden”.
51. Setelah ID dimasukkan, sistem melakukan pencarian data insiden menggunakan fungsi cariInsiden(id) untuk mendapatkan posisi data dalam array.

52. Hasil pencarian disimpan dalam variabel posisi. Jika posisi == -1, maka ID tidak ditemukan atau tidak valid, sehingga sistem menampilkan pesan “ID tidak valid” dan proses berakhir tanpa ada penghapusan data.
53. Jika ID valid (posisi != -1), maka sistem melanjutkan proses dengan menginisialisasi variabel i = posisi sebagai titik awal penghapusan data dalam array.
54. Sistem kemudian melakukan proses pergeseran data menggunakan perulangan dengan kondisi i < panjang - 1.
55. Jika kondisi bernilai true, maka data pada indeks ke-i akan digantikan dengan data pada indeks berikutnya, yaitu data[i] = data[i+1].
56. Setelah proses penyalinan data, nilai indeks i akan ditambahkan satu (i++) untuk melanjutkan pergeseran ke elemen berikutnya.
57. Proses ini akan terus berulang hingga kondisi i < panjang - 1 bernilai false, yang menandakan bahwa seluruh elemen setelah posisi yang dihapus telah digeser ke kiri.
58. Setelah pergeseran selesai, sistem mengurangi jumlah data insiden dengan melakukan panjang-- untuk menghapus elemen terakhir yang sudah tidak digunakan.
59. Sistem kemudian menampilkan pesan “insiden berhasil dihapus” sebagai konfirmasi bahwa proses penghapusan berhasil dilakukan.
60. Proses berakhir (End), dan pengguna dikembalikan ke menu utama.

B. User

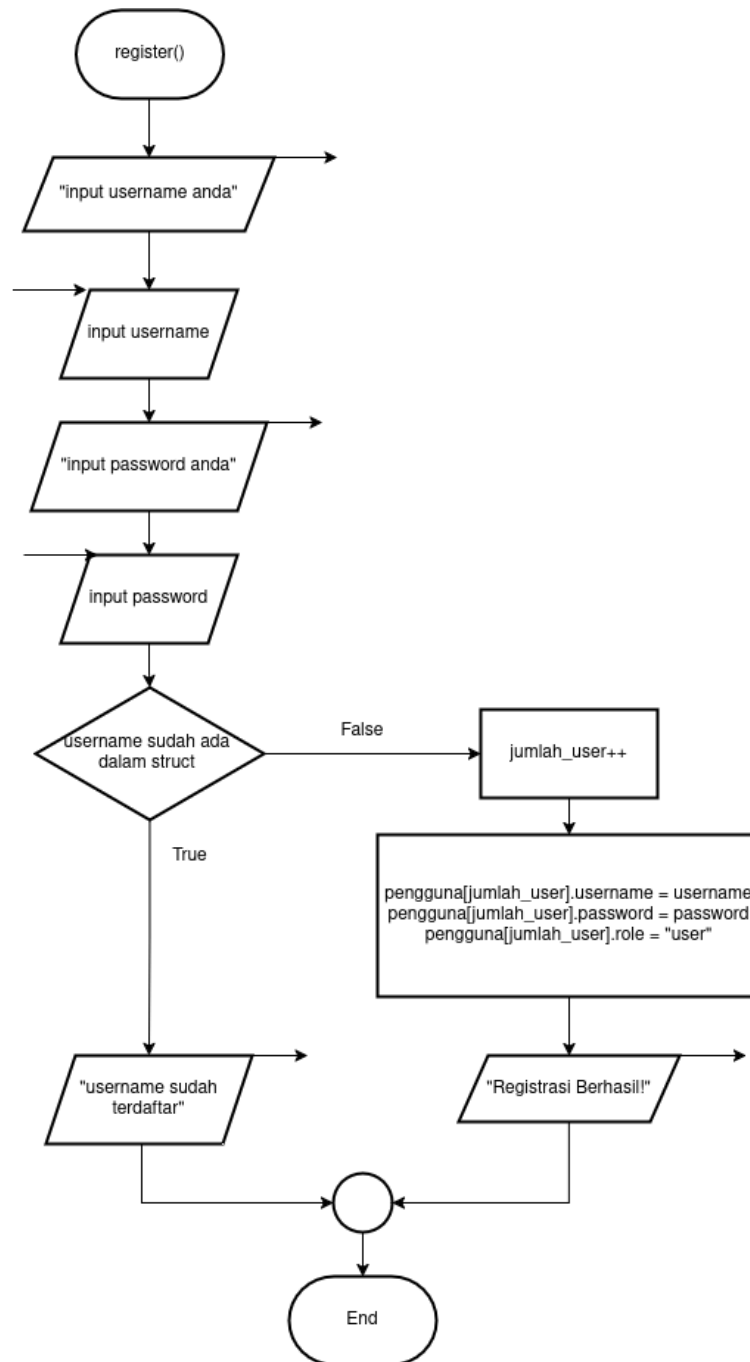
Kurang lebih seperti admin, hanya saja yang menjadi pembeda hanyalah di beberapa bagian dan beberapa fitur seperti fitur login, register, menu utama, lihat insiden dan ubah status.



Gambar 1.11 Login User

61. Proses dimulai dari pemanggilan fungsi login() ketika user memilih menu login pada sistem.
62. Sistem terlebih dahulu meminta user untuk memasukkan username dengan menampilkan pesan "input username anda".

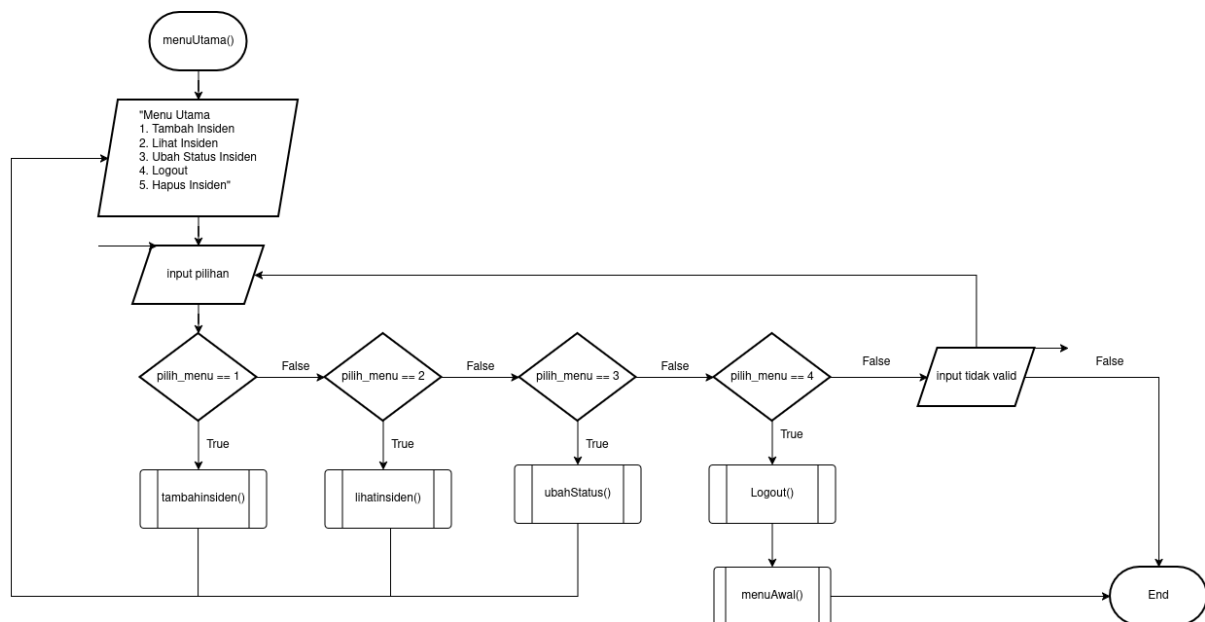
63. User kemudian menginput username. Setelah itu, sistem meminta user untuk memasukkan password dengan menampilkan pesan “input password anda”.
64. User menginput password, dan sistem menyimpan nilai tersebut untuk dilakukan proses validasi.
65. Sistem kemudian melakukan pengecekan kecocokan data login dengan membandingkan username dan password yang diinput dengan data yang tersimpan pada sistem.
66. Jika kondisi username dan password sesuai (valid), maka kondisi bernilai true dan sistem menampilkan pesan “login berhasil!”.
67. Setelah login berhasil, proses berakhir (End) dan user dapat mengakses menu utama sesuai role atau hak aksesnya.
68. Jika username atau password tidak sesuai (kondisi false), maka sistem menampilkan pesan “username atau password salah”.
69. Setelah itu, proses akan kembali ke tahap input username dan password, sehingga user dapat mencoba login kembali hingga data yang dimasukkan benar.



Gambar 1.12 Register

70. Proses dimulai dari pemanggilan fungsi register() ketika user memilih menu untuk melakukan pendaftaran akun baru.
71. Sistem terlebih dahulu meminta user untuk memasukkan username dengan menampilkan pesan “input username anda” dan user kemudian menginput username.
72. Setelah itu, sistem meminta user untuk memasukkan password dengan menampilkan pesan “input password anda” dan user pun diminta menginput password.

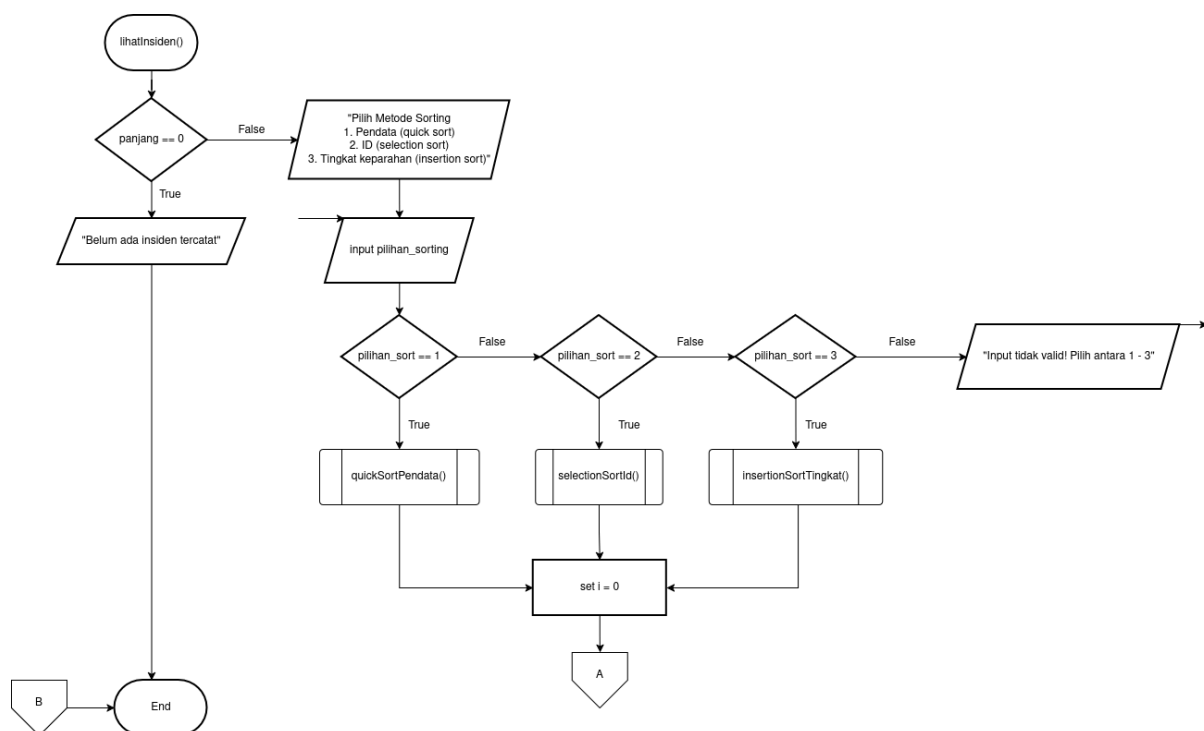
73. Sistem kemudian melakukan pengecekan apakah username yang dimasukkan sudah ada di dalam data (struct pengguna).
74. Jika username sudah terdaftar (kondisi true), maka sistem menampilkan pesan “username sudah terdaftar” dan proses berakhir tanpa menambahkan data baru.
75. Jika username belum terdaftar (kondisi false), maka sistem melanjutkan proses dengan menambahkan jumlah user melalui operasi `jumlah_user++`.
76. Setelah itu, sistem menyimpan data user ke dalam array pengguna dengan mengisi atribut username, password, dan role (sebagai "user").
77. Sistem kemudian menampilkan pesan “Registrasi Berhasil!” sebagai konfirmasi bahwa proses pendaftaran berhasil dilakukan.
78. Proses berakhir (End), dan user dapat kembali ke menu utama untuk melakukan login.



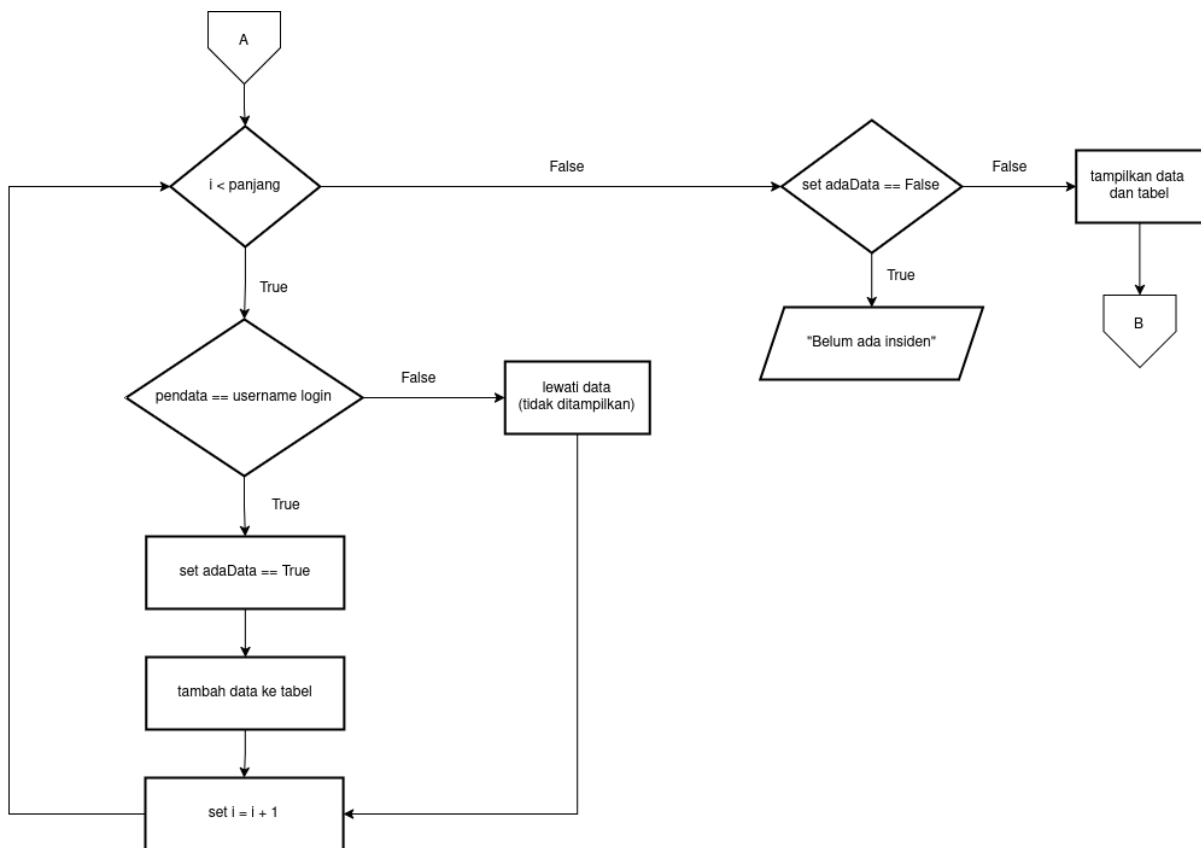
Gambar 1.13 Menu Utama User

79. Proses dimulai dari pemanggilan fungsi `menuUtama()` setelah user berhasil login ke dalam sistem.
80. Sistem kemudian menampilkan daftar menu utama yang dapat diakses oleh user, yaitu: (1) Tambah Insiden, (2) Lihat Insiden, (3) Ubah Status Insiden, (4) Logout.
81. Setelah menu ditampilkan, sistem meminta user untuk memasukkan pilihan menu melalui `input pilih_menu`.
82. Sistem kemudian melakukan pengecekan terhadap nilai input yang dimasukkan oleh user.

83. Jika user memilih menu 1 ($\text{pilih_menu} == 1$), maka sistem akan menjalankan fungsi `tambahinsiden()` untuk menambahkan data insiden baru.
84. Jika user memilih menu 2 ($\text{pilih_menu} == 2$), maka sistem akan menjalankan fungsi `lihatinsiden()` untuk menampilkan daftar insiden yang tersedia.
85. Jika user memilih menu 3 ($\text{pilih_menu} == 3$), maka sistem akan menjalankan fungsi `ubahStatus()` untuk mengubah status insiden.
86. Jika user memilih menu 4 ($\text{pilih_menu} == 4$), maka sistem akan menjalankan fungsi `logout()` untuk keluar dari sistem.
87. Jika input yang dimasukkan tidak sesuai dengan pilihan menu yang tersedia (bukan 1–4), maka sistem menampilkan pesan “input tidak valid”.
88. Kemudian sistem akan kembali ke proses input pilihan menu sampai user memasukkan pilihan yang benar.
89. Proses ini akan terus berulang selama user masih dalam kondisi login, sehingga user dapat terus menggunakan fitur yang tersedia.



Gambar 1.14 Lihat Insiden (User)



Gambar 1.15 Lihat Insiden (User)

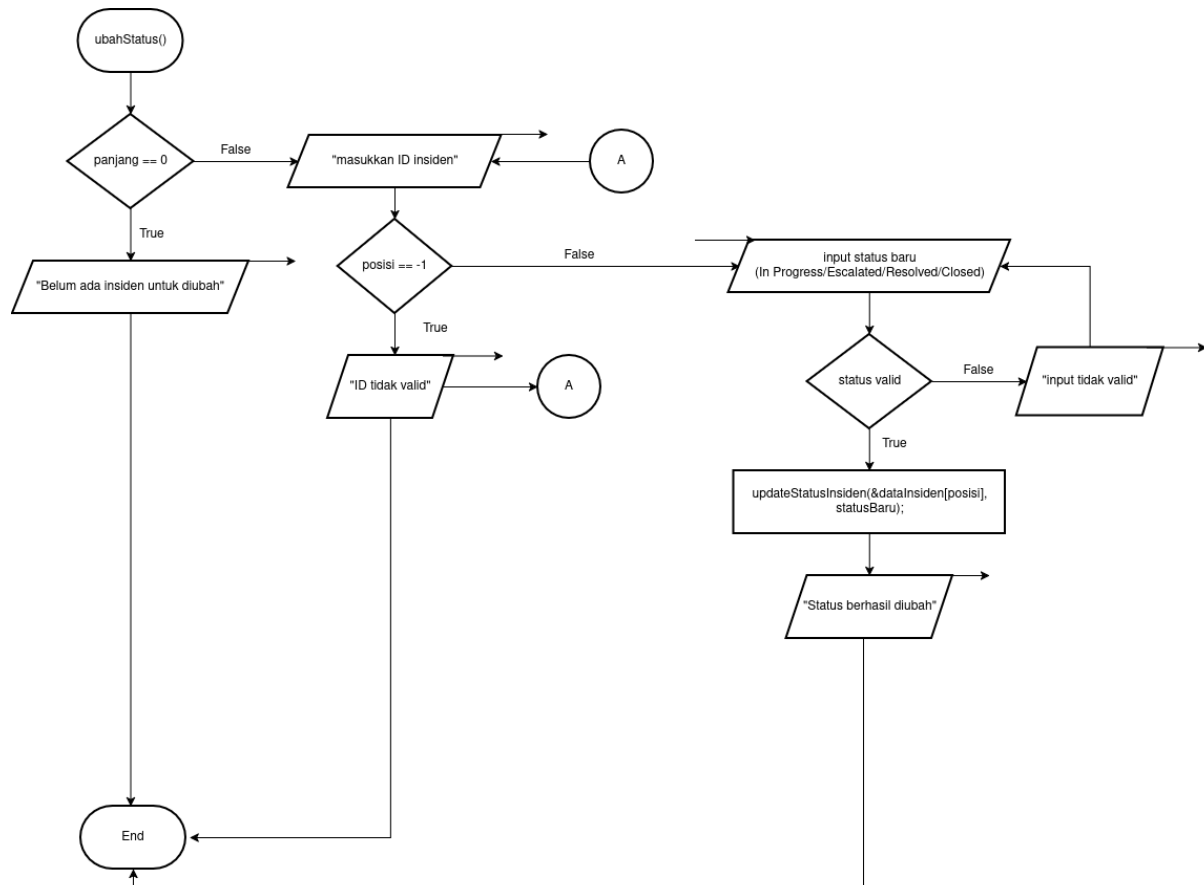
90. Proses dimulai dari pemanggilan fungsi lihatInsiden() ketika user memilih menu untuk melihat data insiden. Pada tahap awal, sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan memeriksa kondisi panjang == 0. Jika kondisi bernilai true, berarti belum ada data insiden yang tersimpan di dalam sistem, sehingga sistem menampilkan pesan “Belum ada insiden tercatat”, lalu proses berakhir.
91. Jika kondisi panjang == 0 bernilai false, maka sistem melanjutkan proses dengan menampilkan pilihan metode sorting kepada user. Pilihan yang tersedia adalah: (1) Pendata (Quick Sort), (2) ID (Selection Sort), (3) Tingkat Keparahannya (Insertion Sort). Setelah pilihan ditampilkan, sistem meminta user untuk memasukkan input untuk pilihan_sorting.
92. Berikutnya, sistem memeriksa nilai input tersebut melalui beberapa percabangan. Jika pilihan_sort == 1, maka sistem menjalankan fungsi quickSortPendata() untuk mengurutkan data berdasarkan pendata. Jika pilihan_sort == 2, maka sistem menjalankan fungsi selectionSortId() untuk mengurutkan data berdasarkan ID. Jika pilihan_sort == 3, maka sistem menjalankan fungsi insertionSortTingkat() untuk

mengurutkan data berdasarkan tingkat keparahan. Namun, jika input tidak sesuai dengan ketiga pilihan tersebut, maka sistem menampilkan pesan “Input tidak valid! Pilih antara 1 - 3”.

93. Setelah salah satu proses sorting selesai dijalankan, sistem menginisialisasi variabel $i = 0$ untuk memulai proses penelusuran data insiden satu per satu. Selanjutnya, alur berpindah ke bagian lanjutan melalui off-page A. Pada bagian ini, sistem memeriksa kondisi $i < \text{panjang}$ untuk menentukan apakah masih ada data yang perlu diperiksa. Jika kondisi bernilai *true*, maka sistem melanjutkan ke proses pengecekan data berdasarkan user yang sedang login. Jika kondisi bernilai *false*, maka sistem berpindah ke pemeriksaan berikutnya, yaitu kondisi $\text{adaData} == \text{False}$.
94. Selama kondisi $i < \text{panjang}$ bernilai *true*, sistem memeriksa apakah $\text{pendata} == \text{username_login}$. Kondisi ini bertujuan untuk memastikan bahwa data yang sedang diperiksa memang milik user yang sedang login. Jika kondisi bernilai *true*, maka sistem menetapkan $\text{adaData} = \text{True}$, lalu menambahkan data tersebut ke dalam tabel. Setelah itu nilai i dinaikkan dengan perintah $i = i + 1$, kemudian proses kembali ke pengecekan $i < \text{panjang}$ untuk memeriksa data berikutnya.
95. Sebaliknya, jika kondisi $\text{pendata} == \text{username_login}$ bernilai *false*, maka data tersebut tidak ditampilkan karena bukan milik user yang sedang login. Dalam flowchart, langkah ini ditunjukkan dengan proses “lewati data (tidak ditampilkan)”. Setelah data dilewati, nilai i tetap dinaikkan dengan perintah $i = i + 1$, lalu sistem kembali ke pengecekan $i < \text{panjang}$ untuk melanjutkan iterasi ke data berikutnya.
96. Setelah seluruh data selesai diperiksa dan kondisi $i < \text{panjang}$ bernilai *false*, sistem memeriksa kondisi $\text{adaData} == \text{False}$. Kondisi ini digunakan untuk mengetahui apakah selama proses perulangan terdapat data milik user yang berhasil ditemukan atau tidak. Jika kondisi bernilai *true*, berarti tidak ada satupun data insiden yang sesuai dengan username_login , sehingga sistem menampilkan pesan “Belum ada insiden”.
97. Namun, jika kondisi $\text{adaData} == \text{False}$, berarti terdapat data insiden milik user yang berhasil ditemukan dan dimasukkan ke dalam tabel. Dalam kondisi ini, sistem menampilkan data dan tabel ke layar. Setelah itu alur berpindah melalui off-page B lalu kembali ke menu utama, kemudian flowchart berakhir pada End.
98. Secara keseluruhan, flowchart `lihatInsiden()` versi user menggambarkan bahwa sistem hanya akan menampilkan data insiden yang sesuai dengan username user yang sedang login. Meskipun data insiden di sistem ada, jika tidak ada satupun data dengan

pendata yang sama dengan username_login, maka user tetap akan menerima output “Belum ada insiden”. Dengan demikian, proses ini memastikan bahwa user biasa hanya dapat melihat data insiden miliknya sendiri.

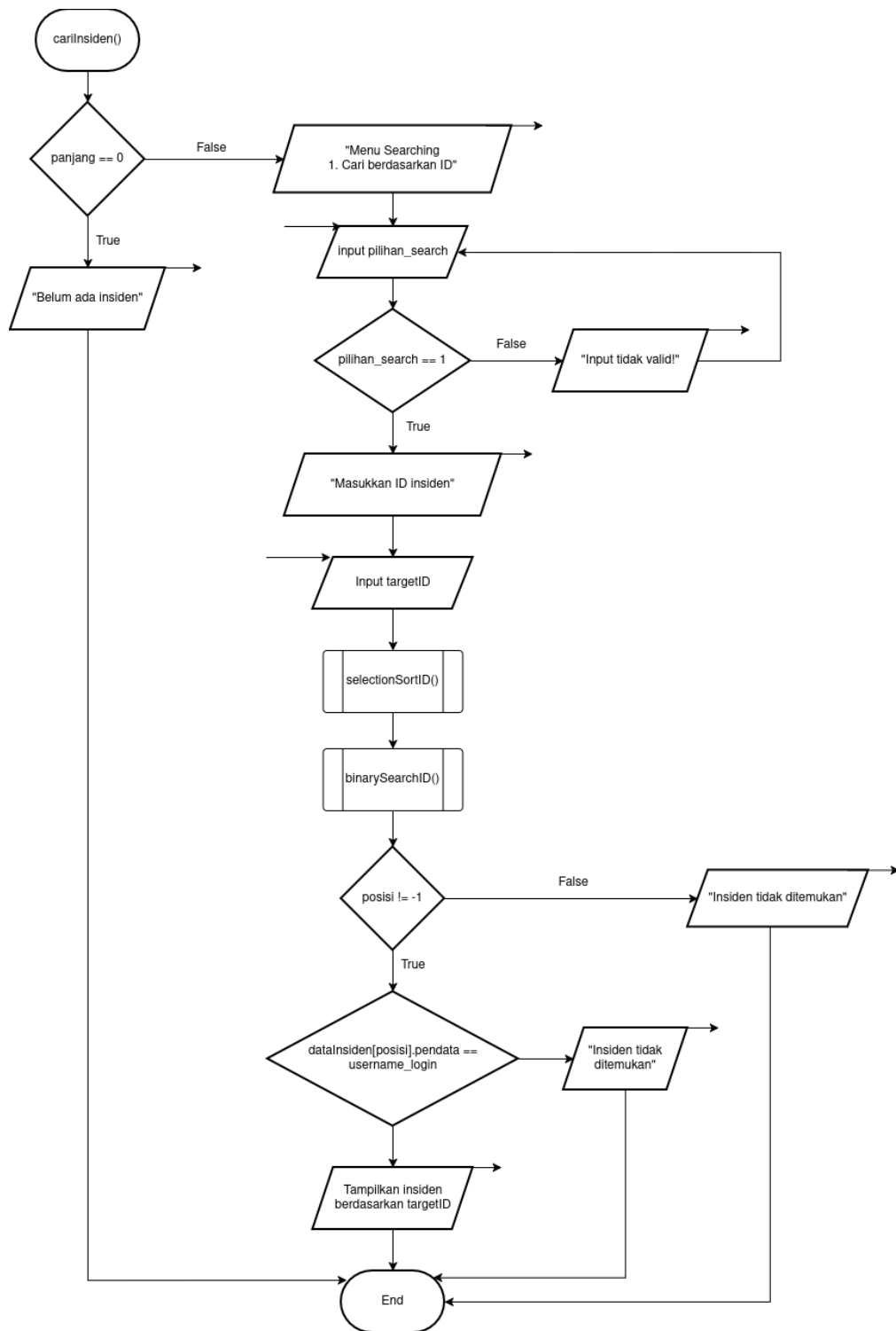
99. Setelah seluruh proses selesai, sistem mengakhiri proses (End) dan user dikembalikan ke menu utama.



Gambar 1.16 Ubah Status (User)

100. Proses dimulai dari pemanggilan fungsi ubahStatus() ketika user memilih menu untuk mengubah status insiden.
101. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel panjang. Jika panjang == 0, maka kondisi bernilai true dan sistem menampilkan pesan “Belum ada insiden untuk diubah”, kemudian proses berakhir tanpa melakukan perubahan.
102. Jika terdapat data insiden (panjang != 0), maka kondisi bernilai false dan sistem melanjutkan proses dengan meminta user untuk memasukkan ID insiden melalui pesan “masukkan ID insiden”.

103. Setelah ID dimasukkan, sistem melakukan pencarian data insiden untuk menentukan posisi data.
104. Jika hasil pencarian menunjukkan posisi == -1, maka ID tidak valid sehingga sistem menampilkan pesan “ID tidak valid” dan user diminta untuk menginput ulang ID hingga benar.
105. Jika ID valid (posisi != -1), maka sistem melanjutkan dengan meminta user untuk memasukkan status baru dengan pilihan In Progress, Escalated, Resolved, Closed.
106. Sistem kemudian melakukan validasi terhadap status yang diinput oleh user.
107. Jika status yang dimasukkan tidak sesuai dengan pilihan yang tersedia, maka sistem menampilkan pesan “input tidak valid” dan user diminta untuk menginput ulang status hingga valid.
108. Jika status yang dimasukkan valid, maka sistem melakukan proses pembaruan dengan mengubah nilai data insiden.
109. Setelah proses update berhasil, sistem menampilkan pesan “Status berhasil diubah” sebagai konfirmasi kepada user.
110. Proses kemudian berakhir (End), dan user dikembalikan ke menu utama untuk melanjutkan aktivitas.



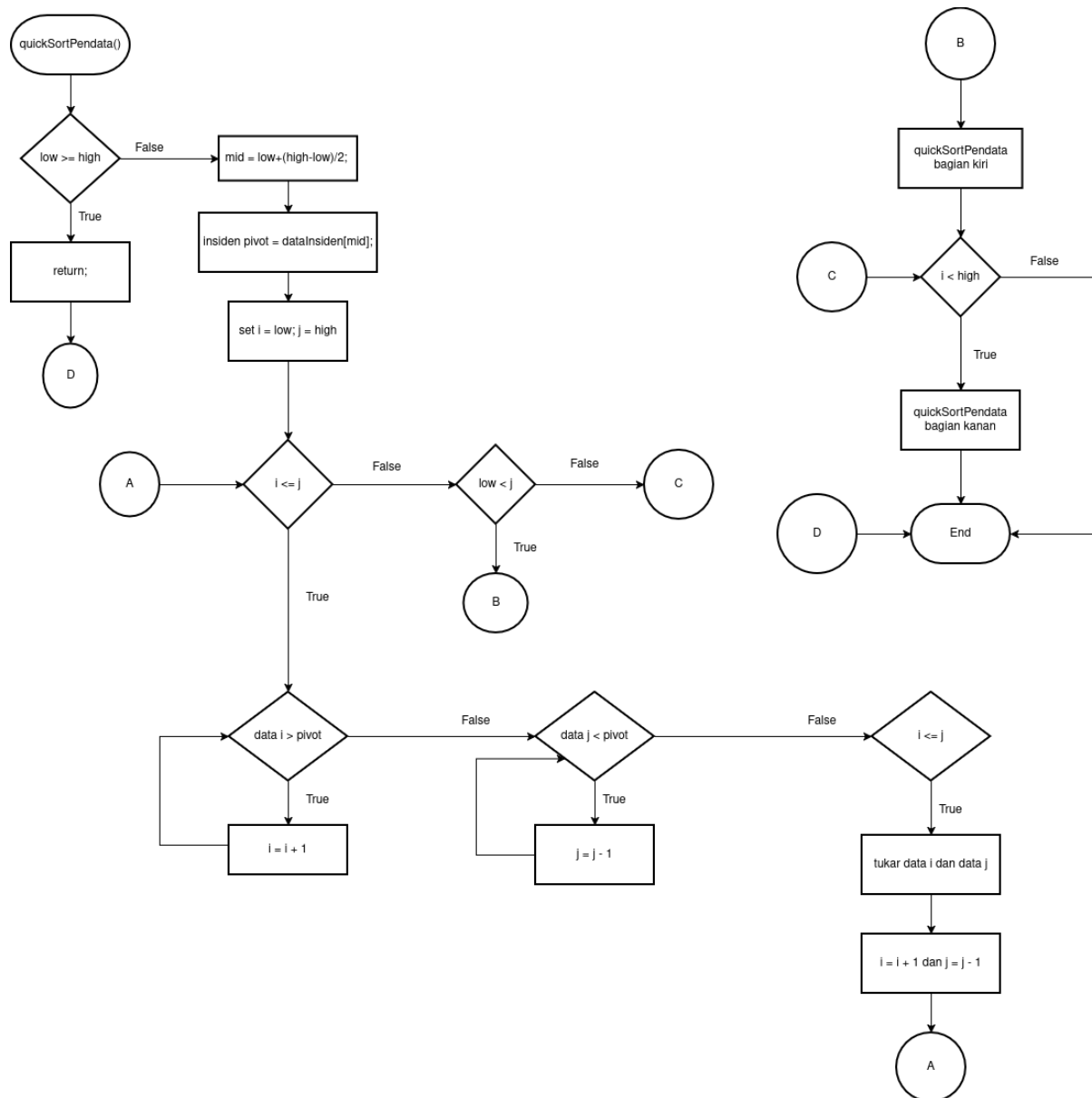
Gambar 1.17 Fungsi Cari Insiden (User)

111. Pada fungsi cariInsiden(), proses dimulai ketika sistem memeriksa apakah jumlah data insiden sama dengan nol ($\text{panjang} == 0$). Jika tidak ada data insiden yang tersimpan, maka sistem akan menampilkan pesan “Belum ada insiden” dan proses pencarian selesai. Sebaliknya, jika data insiden tersedia, maka sistem akan

menampilkan menu pencarian kepada pengguna. Pada proses ini, user melakukan pencarian insiden berdasarkan ID insiden yang ingin dicari.

112. Setelah menu pencarian ditampilkan, pengguna diminta untuk memasukkan pilihan pencarian ke dalam variabel `pilihan_search`. Sistem kemudian melakukan validasi terhadap input tersebut. Pada proses pencarian khusus user, pilihan yang dapat digunakan adalah cari berdasarkan ID. Jika input yang dimasukkan tidak sesuai, maka sistem akan menampilkan pesan “Input tidak valid!” dan pengguna diminta untuk menginput ulang pilihan sampai benar.
113. Jika pilihan pencarian valid, sistem akan meminta pengguna memasukkan ID insiden yang ingin dicari. Input tersebut disimpan ke dalam variabel `targetID`. Setelah itu, sistem melanjutkan proses pencarian dengan terlebih dahulu mengurutkan data insiden berdasarkan ID menggunakan fungsi `selectionSortId()`. Pengurutan ini dilakukan agar pencarian data dapat dilakukan secara lebih efisien menggunakan metode Binary Search. Setelah data terurut, sistem menjalankan fungsi `binarySearchID()` untuk mencari posisi data insiden berdasarkan ID yang dimasukkan. Hasil pencarian disimpan ke dalam variabel `posisi`.
114. Selanjutnya, sistem memeriksa apakah data insiden dengan ID tersebut berhasil ditemukan. Jika hasil pencarian menunjukkan bahwa `posisi == -1`, berarti insiden dengan ID yang dimasukkan tidak ditemukan, sehingga sistem menampilkan pesan “Insiden tidak ditemukan” dan proses pencarian selesai. Namun, jika nilai `posisi` tidak sama dengan `-1`, maka sistem melanjutkan ke tahap pemeriksaan hak akses terhadap data insiden yang ditemukan.
115. Pada tahap ini, sistem membandingkan nilai `dataInsiden[posisi].pendata` dengan `username_login`. Pemeriksaan ini bertujuan untuk memastikan bahwa data insiden yang ditemukan memang milik pengguna yang sedang login. Jika nilai `dataInsiden[posisi].pendata` sama dengan `username_login`, maka sistem akan menampilkan detail insiden berdasarkan `targetID`. Sebaliknya, jika data insiden tersebut bukan milik pengguna yang sedang login, maka sistem akan menampilkan pesan “Insiden tidak ditemukan”. Dengan demikian, user hanya dapat melihat data insiden yang didata oleh akun miliknya sendiri.

1.2 Flowchart Metode Sorting



Gambar 1.18 Fungsi Quick Sort

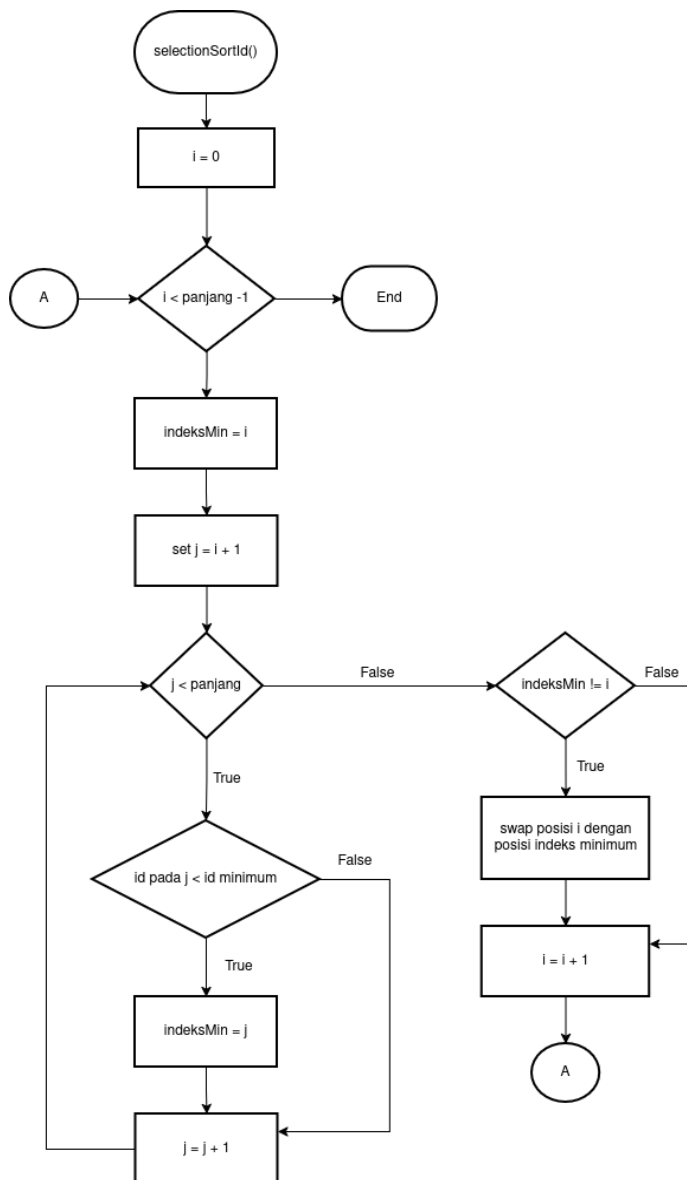
116. Proses dimulai dari pemanggilan fungsi `quickSortPendata(low, high)` ketika user memilih metode pengurutan berdasarkan pendata pada menu lihat insiden. Fungsi ini digunakan untuk mengurutkan data insiden berdasarkan nama pendata secara descending.
117. Sistem terlebih dahulu melakukan pengecekan kondisi `low >= high`. Kondisi ini digunakan untuk menentukan apakah bagian data yang sedang diproses masih perlu diurutkan atau tidak. Jika `low >= high`, maka kondisi bernilai *true* dan fungsi langsung

menjalankan return, sehingga proses pada bagian tersebut selesai karena data sudah dianggap terurut atau hanya tersisa satu elemen.

118. Jika $low \geq high$ bernilai false, maka sistem melanjutkan proses dengan menghitung nilai tengah menggunakan rumus: $mid = low + (high - low) / 2$. Nilai tengah ini digunakan untuk menentukan **pivot**, yaitu elemen acuan dalam proses partisi. Setelah itu sistem menyimpan nilai pivot dengan perintah: **`dataInsiden[mid]`**.
119. Selanjutnya sistem menginisialisasi dua variabel indeks, yaitu: $i = low$ dan $j = high$.
120. Variabel i digunakan untuk menelusuri data dari kiri ke kanan, sedangkan j digunakan untuk menelusuri data dari kanan ke kiri.
121. Setelah itu sistem memeriksa kondisi $i \leq j$. Jika kondisi ini bernilai true, maka proses partisi dilakukan. Pada tahap ini sistem lebih dulu memeriksa apakah data pada posisi i masih lebih besar dari pivot. Karena pengurutan dilakukan secara descending, selama data pada posisi i masih sesuai dengan aturan urutan, maka nilai i akan terus ditambah dengan perintah $i = i + 1$. Proses ini diulang sampai ditemukan data yang tidak sesuai posisi.
122. Setelah proses dari sisi kiri selesai, sistem memeriksa apakah data pada posisi j lebih kecil dari pivot. Jika kondisi ini bernilai true, maka nilai j akan dikurangi dengan perintah $j = j - 1$. Proses ini juga dilakukan berulang sampai ditemukan data yang tidak sesuai posisi dari sisi kanan.
123. Setelah indeks i dan j berhenti pada posisi masing-masing, sistem kembali memeriksa kondisi $i \leq j$. Jika kondisi ini masih bernilai true, maka sistem menukar data pada posisi i dan j . Setelah pertukaran dilakukan, sistem memperbarui kedua indeks dengan: $i = i + 1$ dan $j = j - 1$.
124. Kemudian alur kembali ke pengecekan $i \leq j$ untuk melanjutkan proses partisi sampai kedua indeks saling melewati.
125. Jika kondisi $i \leq j$ bernilai false, berarti proses partisi pada bagian data tersebut telah selesai. Sistem kemudian memeriksa kondisi $low < j$. Jika kondisi ini bernilai true, maka masih ada bagian kiri array yang belum terurut, sehingga sistem memanggil kembali fungsi `quickSortPendata(low, j)` untuk mengurutkan bagian kiri.
126. Setelah proses pada bagian kiri selesai, sistem memeriksa kondisi $i < high$. Jika kondisi ini bernilai true, maka masih ada bagian kanan array yang belum terurut,

sehingga sistem memanggil kembali fungsi `quickSortPendata(i, high)` untuk mengurutkan bagian kanan

127. Jika kondisi $low < j$ bernilai false, maka bagian kiri tidak perlu diproses lagi. Begitu juga jika kondisi $i < high$ bernilai false, maka bagian kanan juga tidak perlu diproses lagi. Setelah kedua pemeriksaan tersebut selesai, maka proses Quick Sort pada bagian data tersebut berakhir.

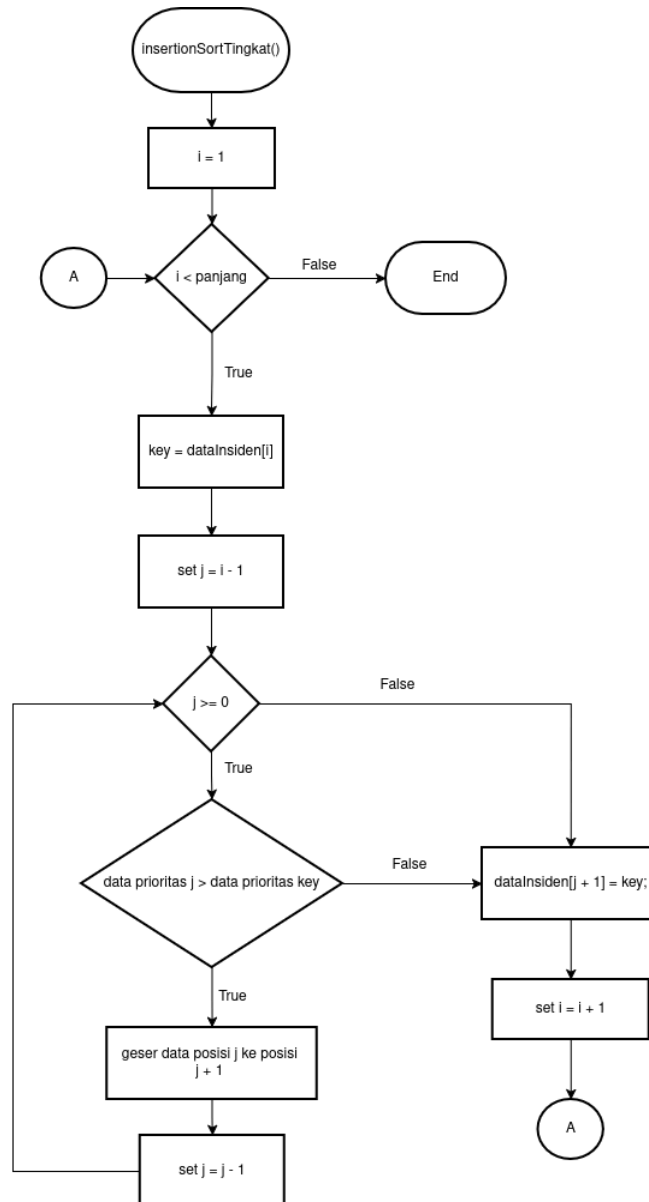


Gambar 1.19 Fungsi Selection Sort

128. Proses dimulai dari pemanggilan fungsi `selectionSortId()` ketika user memilih metode pengurutan berdasarkan ID pada menu lihat insiden. Fungsi ini digunakan untuk mengurutkan data insiden berdasarkan nilai ID secara ascending, yaitu dari nilai terkecil ke nilai terbesar.

129. Pada awal proses, sistem menginisialisasi variabel $i = 0$. Variabel i digunakan sebagai penanda posisi data yang sedang diisi dengan nilai ID terkecil dari bagian data yang belum terurut.
130. Setelah itu sistem memeriksa kondisi $i < \text{panjang} - 1$. Kondisi ini digunakan untuk menentukan apakah masih ada bagian data yang perlu diproses. Jika kondisi bernilai false, berarti seluruh data sudah selesai diurutkan, sehingga proses berakhir. Jika kondisi bernilai true, maka sistem melanjutkan ke langkah berikutnya.
131. Sistem kemudian menetapkan $\text{indeksMin} = i$. Langkah ini berarti data pada posisi i untuk sementara dianggap sebagai data dengan ID paling kecil pada bagian array yang sedang diperiksa. Setelah itu sistem menginisialisasi variabel $j = i + 1$. Variabel j digunakan untuk menelusuri data-data setelah posisi i guna mencari kemungkinan adanya ID yang lebih kecil.
132. Selanjutnya sistem memeriksa kondisi $j < \text{panjang}$. Jika kondisi bernilai true, maka sistem membandingkan ID pada posisi j dengan ID minimum sementara yang berada pada posisi indeksMin . Jika ID pada posisi j lebih kecil dari ID minimum, maka kondisi bernilai true dan sistem memperbarui nilai $\text{indeksMin} = j$. Dengan begitu, posisi minimum berpindah ke indeks yang baru ditemukan.
133. Jika perbandingan menunjukkan bahwa ID pada posisi j tidak lebih kecil dari ID minimum, maka nilai indeksMin tidak berubah. Setelah proses perbandingan selesai, sistem menaikkan nilai j dengan perintah $j = j + 1$, lalu kembali ke pengecekan $j < \text{panjang}$. Langkah ini dilakukan berulang sampai seluruh data di sebelah kanan posisi i selesai diperiksa.
134. Ketika kondisi $j < \text{panjang}$ bernilai false, berarti pencarian nilai minimum pada bagian data tersebut telah selesai. Sistem kemudian memeriksa kondisi $\text{indeksMin} \neq i$. Jika kondisi ini bernilai true, berarti ditemukan data dengan ID yang lebih kecil dari data pada posisi i , sehingga sistem melakukan pertukaran data pada posisi i dengan data pada posisi indeksMin . Tujuan pertukaran ini adalah menempatkan nilai ID terkecil pada posisi yang seharusnya.
135. Jika kondisi $\text{indeksMin} \neq i$ bernilai false, berarti data pada posisi i sudah merupakan data dengan ID terkecil, sehingga pertukaran tidak perlu dilakukan. Setelah itu, sistem menaikkan nilai i dengan perintah $i = i + 1$, lalu kembali ke pengecekan $i < \text{panjang} - 1$ untuk memproses posisi berikutnya.
136. Proses tersebut terus berulang sampai seluruh data tersusun secara terurut. Dengan demikian, algoritma Selection Sort ID bekerja dengan cara mencari nilai ID terkecil

dari bagian data yang belum terurut, lalu menempatkannya pada posisi yang tepat di depan. Setelah itu proses yang sama dilakukan kembali pada sisa data sampai semua data insiden terurut berdasarkan ID dari kecil ke besar.



Gambar 1.20 Fungsi Insertion Sort

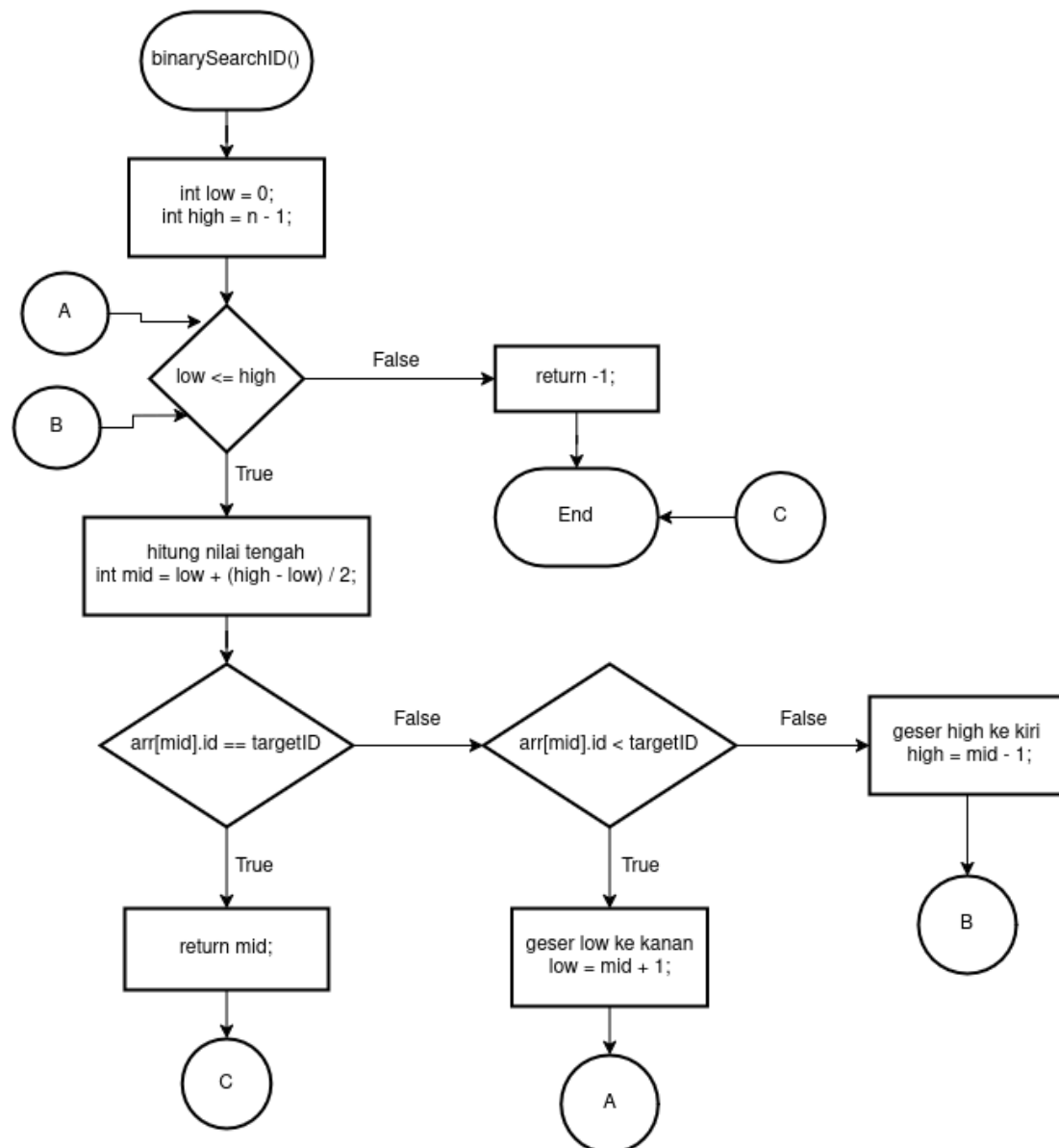
137. Proses dimulai dari pemanggilan fungsi `insertionSortTingkat()` ketika user memilih metode pengurutan berdasarkan tingkat keparahan pada menu lihat insiden. Fungsi ini digunakan untuk mengurutkan data insiden berdasarkan prioritas tingkat keparahan secara ascending, yaitu dari prioritas yang lebih rendah ke prioritas yang lebih tinggi.

138. Pada awal proses, sistem menginisialisasi variabel $i = 1$. Nilai ini dipilih karena elemen pertama dianggap sudah berada pada posisi terurut, sehingga proses penyisipan dimulai dari elemen kedua.
139. Setelah itu sistem memeriksa kondisi $i < \text{panjang}$. Kondisi ini digunakan untuk menentukan apakah masih ada data yang perlu diproses. Jika kondisi bernilai false, berarti seluruh data sudah selesai diurutkan dan proses berakhir. Jika kondisi bernilai true, maka sistem melanjutkan ke langkah berikutnya.
140. Sistem kemudian menyimpan data pada posisi i ke dalam variabel `key` dengan perintah: **`key = dataInsiden[i]`**
141. Variabel `key` berfungsi sebagai data sementara yang akan disisipkan ke posisi yang sesuai dalam bagian data yang sudah terurut. Setelah itu, sistem menginisialisasi variabel $j = i - 1$. Variabel j digunakan untuk menelusuri elemen-elemen di sebelah kiri `key`.
142. Berikutnya sistem memeriksa kondisi $j \geq 0$. Jika kondisi ini bernilai false, berarti tidak ada lagi elemen di sebelah kiri yang perlu dibandingkan, sehingga sistem langsung menempatkan `key` pada posisi $j + 1$. Jika kondisi bernilai true, maka sistem melanjutkan ke proses perbandingan prioritas.
143. Pada tahap ini sistem memeriksa apakah prioritas data pada posisi j lebih besar daripada prioritas `key`. Pemeriksaan ini dilakukan melalui fungsi `prioritasTingkat()`, sehingga yang dibandingkan bukan teks tingkat keparahannya secara langsung, tetapi nilai prioritasnya. Jika kondisi bernilai true, berarti data di sebelah kiri memiliki prioritas yang lebih besar dan harus digeser ke kanan.
144. Ketika kondisi tersebut bernilai true, sistem menggeser data pada posisi j ke posisi $j + 1$. Setelah data digeser, nilai j dikurangi dengan perintah $j = j - 1$. Selanjutnya alur kembali ke pengecekan $j \geq 0$ untuk memeriksa apakah masih ada elemen lain di sebelah kiri yang perlu dibandingkan dengan `key`.
145. Jika kondisi prioritas data j lebih besar dari prioritas `key` bernilai false, berarti posisi yang tepat untuk `key` sudah ditemukan. Dalam kondisi ini, sistem menempatkan `key` ke posisi $j + 1$ dengan perintah: **`dataInsiden[j + 1] = key`**.
146. Setelah `key` ditempatkan pada posisi yang sesuai, sistem menaikkan nilai i dengan perintah $i = i + 1$, lalu kembali ke pengecekan $i < \text{panjang}$ untuk memproses data berikutnya.
147. Proses tersebut berlangsung berulang sampai seluruh data selesai diperiksa dan disisipkan ke posisi yang benar. Dengan demikian, algoritma Insertion Sort Tingkat

Keparahan bekerja dengan cara mengambil satu data sebagai key, lalu membandingkannya dengan data-data sebelumnya. Jika ditemukan data sebelumnya yang memiliki prioritas lebih besar, maka data tersebut digeser ke kanan. Setelah posisi yang tepat ditemukan, key disisipkan pada posisi tersebut. Proses ini diulang sampai seluruh data insiden tersusun berdasarkan tingkat keparahan.

1.3 Flowchart Metode Searching

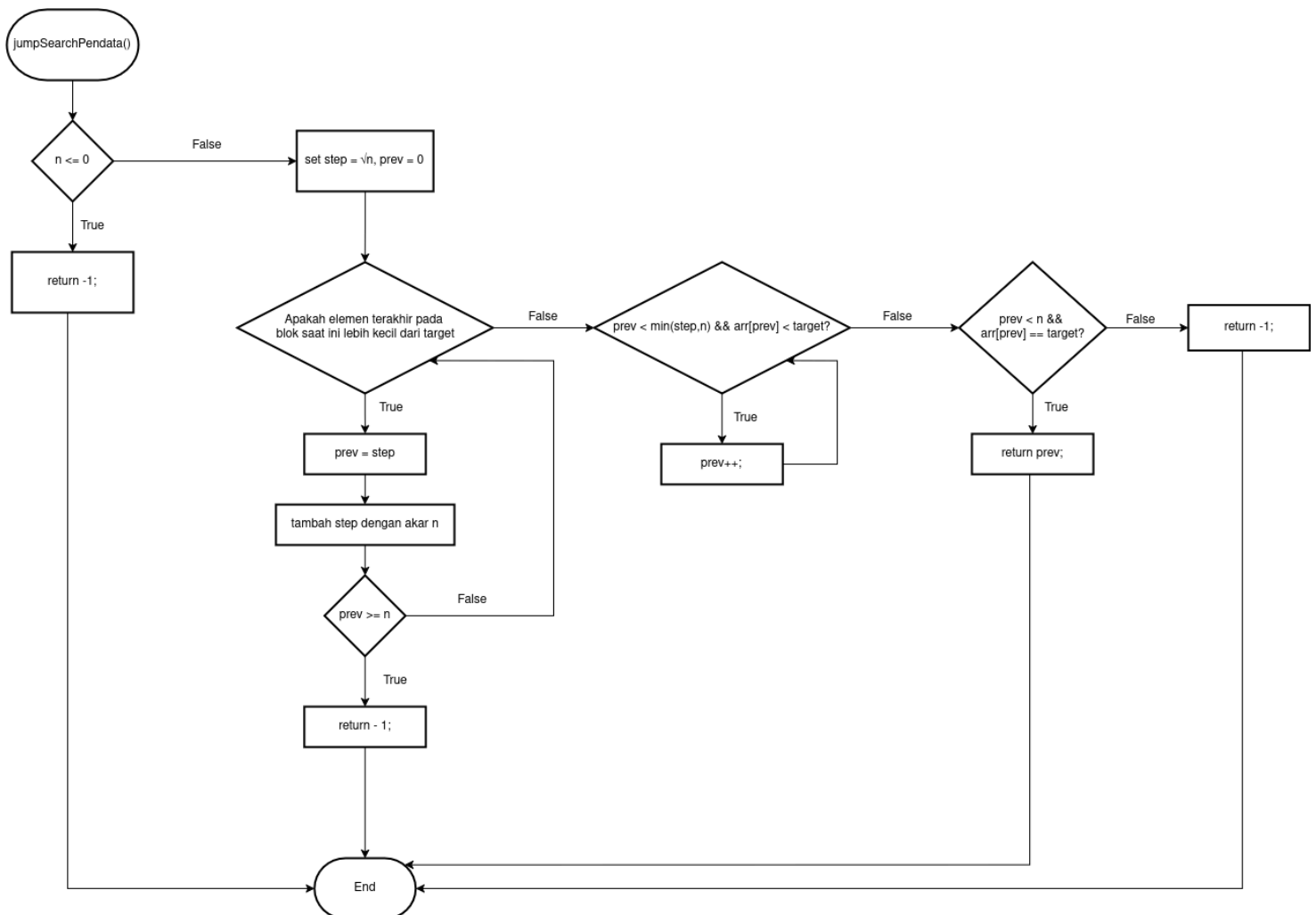
A. Binary Search



Gambar 1.21 Fungsi Binary Search Untuk Mencari ID

148. Pada flowchart `binarySearchID()`, proses dimulai dengan inisialisasi dua variabel, yaitu `low = 0` dan `high = n - 1`. Variabel `low` digunakan untuk menandai batas awal pencarian, sedangkan `high` digunakan untuk menandai batas akhir pencarian pada array data insiden. Dengan demikian, proses pencarian akan dilakukan pada seluruh elemen array dari indeks pertama sampai indeks terakhir.
149. Setelah inisialisasi dilakukan, sistem memeriksa kondisi `low <= high`. Kondisi ini digunakan untuk menentukan apakah ruang pencarian masih tersedia. Jika kondisi tersebut bernilai salah, berarti batas bawah pencarian sudah melewati batas atas, sehingga data yang dicari tidak ditemukan. Dalam keadaan ini, fungsi akan langsung mengembalikan nilai `-1` sebagai tanda bahwa ID insiden yang dicari tidak ada.
150. Namun, jika kondisi `low <= high` bernilai *true*, maka sistem melanjutkan proses dengan menghitung indeks tengah. Nilai tengah dihitung menggunakan rumus: **`mid = low + (high - low) / 2`**. Perhitungan ini dilakukan untuk mendapatkan posisi elemen tengah dari ruang pencarian saat ini. Elemen pada posisi tengah inilah yang kemudian dibandingkan dengan `targetID`.
151. Setelah nilai tengah diperoleh, sistem memeriksa apakah `arr[mid].id == targetID`. Jika kondisi ini terpenuhi, berarti ID yang dicari berhasil ditemukan tepat pada posisi tengah dan mendapatkan kondisi *best case*. Dalam keadaan ini, fungsi akan mengembalikan nilai `mid`, yaitu indeks tempat data ditemukan, dan proses pencarian selesai.
152. Jika `arr[mid].id != targetID`, maka sistem melanjutkan dengan membandingkan apakah `arr[mid].id < targetID`. Apabila kondisi ini bernilai *true*, berarti nilai ID yang dicari berada di sebelah kanan elemen tengah, sehingga batas bawah pencarian digeser ke kanan dengan perintah `low = mid + 1`. Setelah itu, proses kembali mengulangi pemeriksaan kondisi `low <= high` untuk melanjutkan pencarian pada bagian kanan.
153. Sebaliknya, jika `arr[mid].id > targetID`, maka data yang dicari berada di sebelah kiri elemen tengah. Oleh karena itu, batas atas pencarian digeser ke kiri dengan perintah `high = mid - 1`. Setelah batas atas diperbarui, proses kembali ke langkah pemeriksaan `low <= high` agar pencarian dilanjutkan pada bagian kiri array.
154. Proses ini akan terus berulang sampai salah satu dari dua kemungkinan terjadi. Kemungkinan pertama, ID yang dicari ditemukan sehingga fungsi mengembalikan indeks `mid`. Kemungkinan kedua, ruang pencarian habis karena `low` sudah lebih besar dari `high`, sehingga fungsi mengembalikan `-1`. Dengan demikian, algoritma Binary

Search bekerja dengan cara memperkecil ruang pencarian secara bertahap dengan membagi array menjadi dua bagian pada setiap iterasi.



Gambar 1.22 Flowchart Fungsi Jump Search untuk Mencari Berdasarkan Pendata

155. Pada flowchart `jumpSearchPendata()`, proses dimulai dengan memeriksa apakah jumlah data n kurang dari atau sama dengan nol. Pemeriksaan ini dilakukan untuk memastikan bahwa data insiden tersedia sebelum proses pencarian dijalankan. Jika nilai $n \leq 0$, berarti tidak ada data yang dapat dicari, sehingga fungsi langsung mengembalikan nilai -1 sebagai tanda bahwa data tidak ditemukan dan proses selesai.
156. Jika jumlah data lebih dari nol, maka sistem melanjutkan proses dengan melakukan inisialisasi variabel. Pada tahap ini, nilai `step` diatur sebesar $\sqrt{n}$ atau akar dari jumlah data, sedangkan variabel `prev` diatur bernilai 0. Nilai `step` digunakan sebagai ukuran lompatan dalam proses Jump Search, sementara `prev` digunakan untuk menandai posisi awal blok data yang sedang diperiksa.

157. Setelah inisialisasi dilakukan, sistem memeriksa apakah elemen terakhir pada blok saat ini masih lebih kecil daripada data target yang dicari. Pemeriksaan ini dilakukan melalui kondisi `arr[min(step, n) - 1].pendata < targetPendata`. Jika kondisi tersebut bernilai benar, berarti target belum berada pada blok saat ini, sehingga pencarian harus dilanjutkan ke blok berikutnya. Dalam hal ini, nilai `prev` diisi dengan `step`, kemudian nilai `step` ditambahkan lagi dengan `sqrt(n)` untuk memperluas jangkauan blok berikutnya.
158. Selanjutnya, sistem memeriksa apakah nilai `prev` sudah lebih besar atau sama dengan `n`. Jika kondisi ini terpenuhi, berarti pencarian telah melewati batas akhir data, sehingga dapat disimpulkan bahwa pendata yang dicari tidak ditemukan. Dalam kondisi tersebut, fungsi akan mengembalikan nilai `-1` dan proses selesai. Namun, jika `prev` masih berada dalam rentang data, maka proses kembali ke pemeriksaan blok sampai ditemukan blok yang kemungkinan berisi target.
159. Apabila blok telah mendekati posisi target atau `high` telah melebihi target, sistem akan melanjutkan ke tahap pencarian linear di dalam blok tersebut. Pada tahap ini, sistem memeriksa kondisi `prev < min(step, n) && arr[prev].pendata < targetPendata`. Selama kondisi ini bernilai benar, berarti posisi `prev` masih berada dalam batas blok dan nilai pendata pada posisi tersebut masih lebih kecil dari target, sehingga `prev` akan ditambah satu per satu menggunakan operasi `prev++`. Proses ini dilakukan agar sistem dapat menelusuri elemen-elemen dalam blok secara berurutan sampai target ditemukan atau sampai batas blok tercapai.
160. Setelah proses penelusuran linear selesai, sistem melakukan pemeriksaan akhir, yaitu apakah `prev < n` dan `arr[prev].pendata == targetPendata`. Jika kedua kondisi tersebut bernilai benar, maka pada akhir proses pencarian indeks yang sedang ditunjuk oleh `prev` adalah salah satu posisi target, sehingga fungsi mengembalikan `prev`. Sebaliknya, jika kondisi tersebut tidak terpenuhi, maka fungsi mengembalikan nilai `-1` sebagai tanda bahwa pendata yang dicari tidak ditemukan di dalam array.

2. Deskripsi Singkat Program

Program ini dapat mempermudah proses perhitungan konversi satuan panjang tanpa perlu menghitung secara manual. Dalam segi pengembang program ini membantu memahami konsep dasar algoritma, flowchart, variabel, percabangan, dan perulangan dalam pemrograman.

3. Source Code

A. Rekursif

```
int hitungid(string user, int index){
    if(index == panjang)
        return 0;

    if(dataInsiden[index].pendata == user)
        return 1 + hitungid(user, index + 1);

    return hitungid(user, index + 1);
}
```

B. Overloading

```
int cariInsiden(int id){
    for(int i = 0; i < panjang; i++)
        if(dataInsiden[i].id == id)
            return i;
    return -1;
}

int cariInsiden(int id, string user){
    for(int i = 0; i < panjang; i++)
        if(dataInsiden[i].id_user == id && dataInsiden[i].pendata == user)
            return i;
    return -1;
}
```

C. Tambah Insiden

```
void tambahInsiden(string username_login){
    try{
        if(panjang >= max_insiden){
            throw overflow_error("Kapasitas insiden penuh!");
        }

        cout << "Tambah Insiden" << endl;

        string kejadian = input("Laporan Kejadian: ");
    }
```

```

        string tingkat_keparahan;

        while (true) {
            try {
                tingkat_keparahan = input("Tingkat Keparahan
(Critical/High/Medium/Low): ");

                if (tingkat_keparahan != "Critical" &&
                    tingkat_keparahan != "High" &&
                    tingkat_keparahan != "Medium" &&
                    tingkat_keparahan != "Low") {
                    throw invalid_argument("Input tidak valid!");
                }

                break;
            }
            catch (const exception& e) {
                cout << "Error: " << e.what() << endl;
            }
        }

        string tanggal = input("Tanggal: ");

        int id_user = hitungid(username_login, 0) + 1;

        dataInsiden[panjang].id = id_counter;
        dataInsiden[panjang].id_user = id_user;
        dataInsiden[panjang].kejadian = kejadian;
        dataInsiden[panjang].tingkat = tingkat_keparahan;
        dataInsiden[panjang].status = "open";
        dataInsiden[panjang].tanggal = tanggal;
        dataInsiden[panjang].pendata = username_login;

        panjang++;
        id_counter++;

        cout << "Insiden berhasil ditambahkan!" << endl;
    }
    catch (const exception& e){
        cout << "Error: " << e.what() << endl;
    }
}

```

D. Lihat Insiden

```

void lihatInsiden(string username_login, string role){
    try {
        if(panjang == 0){
            throw runtime_error("Belum ada insiden!");
        }
    }
}

```

```

cout << "\n=== Pilih Metode Sorting ===" << endl;
cout << "1. Pendata (Quick Sort)" << endl;
cout << "2. ID (Selection Sort)" << endl;
cout << "3. Tingkat Keparahan (Insertion Sort)" << endl;

int pilihan_sort;

while (true) {
    try {
        pilihan_sort = inputAngka("Pilih sorting: ");

        if (pilihan_sort < 1 || pilihan_sort > 3) {
            throw out_of_range("Pilihan Sorting harus di antara 1 - 3!");
        }

        break;
    }
    catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }
}

switch(pilihan_sort){
    case 1:
        quickSortPendata(0, panjang - 1);
        break;
    case 2:
        selectionSortId();
        break;
    case 3:
        insertionSortTingkat();
        break;
}

bool adaData = false;
for(int i = 0; i < panjang; i++){
    if(role == "admin" || dataInsiden[i].pendata == username_login){
        adaData = true;
        break;
    }
}

if(!adaData){
    throw runtime_error("Belum ada insiden untuk ditampilkan!");
}

Table tabel;
tabel.add_row({"ID", "Kejadian", "Tingkat", "Status", "Tanggal", "Pendata"});

for(int i = 0; i < panjang; i++){

```



```

        if(role == "admin" || dataInsiden[i].pendata == username_login){
            tabel.add_row({
                to_string(dataInsiden[i].id),
                dataInsiden[i].kejadian,
                dataInsiden[i].tingkat,
                dataInsiden[i].status,
                dataInsiden[i].tanggal,
                dataInsiden[i].pendata
            });
        }
    }

    cout << tabel << endl;
}
catch (const exception& e) {
    cout << "Error: " << e.what() << endl;
}
}

```

E. Ubah Status Insiden

```

void ubahStatus(string username_login, string role){
    try {
        if(panjang <= 0){
            throw runtime_error("Belum ada Insiden untuk diubah");
        }

        int posisi;
        int id = inputAngka("Silakan Masukkan ID Insiden: ");

        if(role == "admin")
            posisi = cariInsiden(id);
        else
            posisi = cariInsiden(id, username_login);

        if(posisi == -1) {
            throw runtime_error("ID tidak valid!");
        }

        string statusBaru;

        cout << "Status baru (In Progress/Escalated/Resolved/Closed): ";
        getline(cin, statusBaru);

        if(statusBaru != "In Progress" &&
            statusBaru != "Escalated" &&
            statusBaru != "Resolved" &&
            statusBaru != "Closed"){
            throw invalid_argument("Input tidak valid!");
        }
    }
}

```

```

    }

    updateStatusInsiden(&dataInsiden[posisi], statusBaru);

    cout << "Status berhasil diubah" << endl;
}
catch (const exception& e) {
    cout << "Error: " << e.what() << endl;
}
}
}

```

F. Cari Insiden

```

void cariInsiden(string username_login, string role) {
    try {
        if (panjang == 0) {
            throw runtime_error("Belum ada insiden");
        }

        cout << "\n=== MENU SEARCHING ===" << endl;
        cout << "1. Cari berdasarkan ID" << endl;
        if (role == "admin") {
            cout << "2. Cari berdasarkan Pendata" << endl;
        }

        int pilihan_search = inputAngka("Pilih menu: ");

        if (role == "admin") {
            if (pilihan_search < 1 || pilihan_search > 2) {
                throw out_of_range("Input tidak valid!");
            }
        }
        else {
            if (pilihan_search != 1) {
                throw out_of_range("Input tidak valid!");
            }
        }

        if (pilihan_search == 1) {
            int targetID = inputAngka("Masukkan ID Insiden: ");

            if (targetID <= 0) {
                throw invalid_argument("ID harus lebih dari 0!");
            }

            selectionSortId();
            int posisi = binarySearchID(dataInsiden, panjang, targetID);

            if (posisi == -1) {
                throw runtime_error("Insiden tidak ditemukan!");
            }
        }
    }
}

```

```

    }

    if (role != "admin" && dataInsiden[posisi].pendata != username_login)
    {
        throw runtime_error("Anda tidak memiliki akses ke insiden
tersebut!");
    }

    Table tabel;
    tabel.add_row({"ID", "Kejadian", "Tingkat", "Status", "Tanggal",
"Pendata"});
    tabel.add_row({
        to_string(dataInsiden[posisi].id),
        dataInsiden[posisi].kejadian,
        dataInsiden[posisi].tingkat,
        dataInsiden[posisi].status,
        dataInsiden[posisi].tanggal,
        dataInsiden[posisi].pendata
    });

    cout << "Insiden Ditemukan!" << endl;
    cout << tabel << endl;
}
else if (pilihan_search == 2 && role == "admin") {
    string targetPendata = input("Silakan Masukkan nama Pendata: ");

    quickSortPendata(0, panjang - 1);
    int posisi = jumpSearchPendata(dataInsiden, panjang, targetPendata);

    if (posisi == -1) {
        throw runtime_error("Pendata tidak ditemukan");
    }

    int i = posisi;

    while (i > 0 && dataInsiden[i - 1].pendata == targetPendata) {
        i--;
    }

    Table tabel;
    tabel.add_row({"ID", "Kejadian", "Tingkat", "Status", "Tanggal",
"Pendata"});

    while (i < panjang && dataInsiden[i].pendata == targetPendata) {
        tabel.add_row({
            to_string(dataInsiden[i].id),
            dataInsiden[i].kejadian,
            dataInsiden[i].tingkat,
            dataInsiden[i].status,
            dataInsiden[i].tanggal,
            dataInsiden[i].pendata

```

```

        });
        i++;
    }

    cout << "Daftar Insiden " << targetPendata << ":" << endl;
    cout << tabel << endl;
}
}
catch (const exception& e) {
    cout << "Error: " << e.what() << endl;
}
}
}

```

G. Hapus Insiden

```

void hapusInsiden(){
    try {
        if(panjang == 0) {
            throw runtime_error("Belum ada insiden untuk dihapus!");
        }

        int index = inputAngka("Masukkan ID insiden: ");

        if (index <= 0) {
            throw invalid_argument("ID harus lebih dari 0!");
        }

        int posisi = cariInsiden(index);

        if(posisi == -1){
            throw runtime_error("ID insiden tidak valid atau tidak ditemukan!");
        }

        for(int i = posisi; i < panjang - 1; i++)
            dataInsiden[i] = dataInsiden[i + 1];

        panjang--;

        cout << "Insiden berhasil dihapus" << endl;
    }
    catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }
}
}

```

H. Menu Utama

```

void menuUtama(string &username_login, string &role){
    int pilih_menu;

```

```

while(login){
    try {
        cout << "\n=== MENU UTAMA ===" << endl;
        cout << "Login sebagai: " << username_login << " (" << role << ")" <<
endl;

        cout << "1. Tambah Insiden" << endl;
        cout << "2. Lihat Insiden" << endl;
        cout << "3. Ubah Status Insiden" << endl;
        cout << "4. Cari Insiden" << endl;
        cout << "5. Logout" << endl;
        if(role == "admin")
            cout << "6. Hapus Insiden" << endl;

        pilih_menu = inputAngka("Pilih menu: ");

        if(role == "admin"){
            if(pilih_menu < 1 || pilih_menu > 6){
                throw invalid_argument("Pilih angka di antara 1 - 6!");
            }
        }
        else{
            if(pilih_menu < 1 || pilih_menu > 5){
                throw invalid_argument("Pilih angka di antara 1 - 5!");
            }
        }

        switch(pilih_menu){
            case 1:
                tambahInsiden(username_login);
                break;

            case 2:
                lihatInsiden(username_login, role);
                break;

            case 3:
                ubahStatus(username_login, role);
                break;

            case 4:
                cariInsiden(username_login, role);
                break;

            case 5:
                login = false;
                username_login = "";
                role = "";
                cout << "Logout berhasil!" << endl;
                break;

```

```

        case 6:
            if (role == "admin") {
                hapusInsiden();
            }
            else {
                throw runtime_error("Akses ditolak! Fitur ini hanya untuk
admin.");
            }
            break;
    }
}
catch (const exception& e) {
    cout << "Error: " << e.what() << endl;
}
}
}

```

I. Main

```

int main(){
    int pilihan;
    string username, password;
    string username_login = "";
    string role = "";

    int percobaan_login = 0;

    while(true){
        try {
            if(!login){
                cout << "\nSISTEM MANAJEMEN INSIDEN (SIEM)" << endl;
                cout << "1. Login" << endl;
                cout << "2. Register" << endl;
                cout << "3. Keluar" << endl;

                pilihan = inputAngka("Pilih menu: ");

                if (pilihan < 1 || pilihan > 3) {
                    throw out_of_range("Silakan pilih menu 1 - 3");
                }
            }

            switch(pilihan){
                case 1:
                    while(percobaan_login < 3 && !login){
                        username = input("Silakan Input Username Anda: ");
                        password = input("Silakan input password Anda: ");

                        if(username == "") {
                            throw invalid_argument("Username tidak boleh

```

```

kosong!");
    }

    if (password == "") {
        throw invalid_argument("Password tidak boleh
kosong!");
    }

    if(loginUser(username, password, role)){
        login = true;
        username_login = username;
        percobaan_login = 0;

        cout << "Login Berhasil!" << endl;
        menuUtama(username_login, role);
        break;
    }
    else{
        percobaan_login++;
        cout << "Username atau password salah" << endl;

        if(percobaan_login >= 3){
            cout << "Gagal login 3 kali, Program berhenti."
<< endl;

            return 0;
        }
    }
}
break;

case 2:
    if(jumlah_user >= max_user){
        throw out_of_range("overflow error!");
    }

    username = input("Input username anda: ");
    password = input("Input password anda: ");

    if(registerUser(username, password))
        cout << "Registrasi Berhasil!" << endl;
    else
        cout << "Username sudah terdaftar" << endl;
    break;

case 3:
    cout << "Terima kasih telah menggunakan program ini" << endl;
    return 0;
}
}
catch (const exception &e) {
    cout << "Error: " << e.what() << endl;
}

```

```

        if (percobaan_login >= 3) {
            return 0;
        }
    }
}

return 0;
}

```

J. Helper

```

bool loginUser(string username, string password, string &role){
    if(username == "diftya" && password == "042"){
        role = "admin";
        return true;
    }

    for(int i = 0; i < jumlah_user; i++){
        if(pengguna[i].username == username && pengguna[i].password == password){
            role = "user";
            return true;
        }
    }

    return false;
}

bool registerUser(string username, string password){
    if(username == "diftya")
        return false;

    for(int i = 0; i < jumlah_user; i++){
        if(pengguna[i].username == username)
            return false;
    }

    pengguna[jumlah_user].username = username;
    pengguna[jumlah_user].password = password;
    pengguna[jumlah_user].role = "user";

    jumlah_user++;
    return true;
}

```

K. Dereference (Parameter Fungsi)

```

void updateStatusInsiden(insiden *data, string statusBaru){
    data->status = statusBaru;
}

```


L. Address-of Operator (Parameter Fungsi)

```
updateStatusInsiden(&dataInsiden[posisi], statusBaru);
```

M. Pointer pada struct

```
insiden *data
```

N. Quick Sort (Descending Nama Pendata Insiden)

```
bool lebihKecilPendata(insiden a, insiden b){  
    return a.pendata > b.pendata;
```

```
}  
bool lebihBesarPendata(insiden a, insiden b){  
    return a.pendata < b.pendata;  
}  
void quickSortPendata(int low, int high){  
    if(low >= high)  
        return;  
  
    int mid = low + (high - low) / 2;  
    insiden pivot = dataInsiden[mid];  
  
    int i = low;  
    int j = high;  
  
    while(i <= j){  
        while(i <= high && lebihKecilPendata(dataInsiden[i], pivot)){  
            i++;  
        }  
  
        while(j >= low && lebihBesarPendata(dataInsiden[j], pivot)){  
            j--;  
        }  
  
        if(i <= j){  
            swap(dataInsiden[i], dataInsiden[j]);  
            i++;  
            j--;  
        }  
    }  
  
    if(low < j)  
        quickSortPendata(low, j);  
  
    if(i < high)  
        quickSortPendata(i, high);  
}
```

O. Selection Sort (Ascending ID)

```
void selectionSortId(){
    for(int i = 0; i < panjang - 1; i++){
        int indeksMin = i;

        for(int j = i + 1; j < panjang; j++){
            if(dataInsiden[j].id < dataInsiden[indeksMin].id){
                indeksMin = j;
            }
        }

        if(indeksMin != i){
            swap(dataInsiden[i], dataInsiden[indeksMin]);
        }
    }
}
```

P. Insertion Sort (Ascending Tingkat Keparahan)

```
void insertionSortTingkat(){
    for(int i = 1; i < panjang; i++){
        insiden key = dataInsiden[i];
        int j = i - 1;

        while(j >= 0 &&
            prioritasTingkat(dataInsiden[j].tingkat) >
            prioritasTingkat(key.tingkat)){
            dataInsiden[j + 1] = dataInsiden[j];
            j--;
        }

        dataInsiden[j + 1] = key;
    }
}
```

Q. Binary Search

```
int binarySearchID(insiden *arr, int n, int targetID) {
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid].id == targetID) {
            return mid;
        }
    }
}
```

```

    else if (arr[mid].id < targetID) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}
return -1;
}

```

R. Jump Search

```

int jumpSearchPendata(insiden *arr, int n, string targetPendata) {
    if (n <= 0) {
        return -1;
    }

    int step = sqrt(n);
    int prev = 0;

    while(arr[min(step, n) - 1].pendata < targetPendata) {
        prev = step;
        step += sqrt(n);

        if(prev >= n) {
            return -1;
        }
    }
    while (prev < min(step, n) && arr[prev].pendata < targetPendata) {
        prev++;
    }
    if (prev < n && arr[prev].pendata == targetPendata) {
        return prev;
    }
    return -1;
}

```

4. Hasil Output

```

SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: diftya
Input Password Anda: 042
Login Berhasil!

```

Gambar 4.1 Output program login berhasil

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: yaya
Input Password Anda: biskuit yaya
Username atau password salah

Input Username Anda: █
```

Gambar 4.2 Output program ketika login salah

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah

Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah

Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah
Gagal login 3 kali, Program berhenti.
```

Gambar 4.2 Output Program ketika login salah (percobaan ≥ 3)

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 2
input username anda: yaya
input password anda : biskuit yaya
Registrasi Berhasil!
```

Gambar 4.3 Output Register Berhasil

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 2
input username anda: yaya
input password anda : rengginang
Username sudah terdaftar
```

Gambar 4.4 Output Register Gagal

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih telah menggunakan program ini
```

Gambar 4.5 Output Menu Keluar Program

```

=== MENU UTAMA ===
Login sebagai: diftya (admin)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
5. Hapus Insiden
Pilih menu: █

```

Gambar 4.6 Output Menu Utama Admin

```

=== MENU UTAMA ===
Login sebagai: yaya (user)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
Pilih menu: █

```

Gambar 4.7 Output Menu utama user

```

Pilih menu: 1
Tambah Insiden
Laporan kejadian: Kebocoran data
Input tingkat keparahan (Critical/High/Medium/Low): Critical
Tanggal: 15 Maret 2026
Insiden berhasil ditambahkan!

```

Gambar 4.8 Output Tambah Insiden

```

Pilih menu: 2
+---+-----+-----+-----+-----+
| ID | Kejadian      | Tingkat | Status | Tanggal      | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran data | Critical | open   | 15 Maret 2026 | yaya    |
+---+-----+-----+-----+-----+

```

Gambar 4.9 Output Lihat Insiden

```
Pilih menu: 2
+---+-----+-----+-----+-----+
| ID | Kejadian      | Tingkat | Status   | Tanggal      | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran data | Critical | Escalated | 15 Maret 2026 | yaya    |
+---+-----+-----+-----+-----+
```

Gambar 4.10 Output Ubah Status Insiden

```
=== MENU UTAMA ===
Login sebagai: yaya (user)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
Pilih menu: 4
Logout berhasil!
```

Gambar 4.11 Output Logout

```
=== MENU UTAMA ===
Login sebagai: diftya (admin)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
5. Hapus Insiden
Pilih menu: 5
Masukkan ID insiden: 1
Insiden berhasil dihapus
```

Gambar 4.12 Output Hapus Insiden (Fitur Admin)

=== Pilih Metode Sorting ===

1. Pendata (Quick Sort)
2. ID (Selection Sort)
3. Tingkat Keparahan (Insertion Sort)

Pilih sorting: 1

ID	Kejadian	Tingkat	Status	Tanggal	Pendata
2	Peretasan Server	High	open	4 April 2026	yaya
3	Hilangnya beberapa data	Medium	open	13 April 2026	fitri
1	Kebocoran Data	Critical	open	7 April 2026	diftya

Gambar 4.13 Output Sorting pada Nama Pendata (Descending Quick Sort)

=== Pilih Metode Sorting ===

1. Pendata (Quick Sort)
2. ID (Selection Sort)
3. Tingkat Keparahan (Insertion Sort)

Pilih sorting: 2

ID	Kejadian	Tingkat	Status	Tanggal	Pendata
1	Kebocoran Data	Critical	open	7 April 2026	diftya
2	Peretasan Server	High	open	4 April 2026	yaya
3	Hilangnya beberapa data	Medium	open	13 April 2026	fitri

Gambar 4.14 Output Sorting pada ID (Ascending Selection Sort)

=== Pilih Metode Sorting ===

1. Pendata (Quick Sort)
2. ID (Selection Sort)
3. Tingkat Keparahan (Insertion Sort)

Pilih sorting: 3

ID	Kejadian	Tingkat	Status	Tanggal	Pendata
3	Hilangnya beberapa data	Medium	open	13 April 2026	fitri
2	Peretasan Server	High	open	4 April 2026	yaya
1	Kebocoran Data	Critical	open	7 April 2026	diftya

Gambar 4.15 Output Sorting pada Tingkat Keparahan (Ascending Insertion Sort)


```
=== MENU SEARCHING ===
1. Cari berdasarkan ID
2. Cari berdasarkan Pendata
Pilih menu: 1
Masukkan ID Insiden: 1
Insiden Ditemukan!

+---+-----+-----+-----+-----+
| ID | Kejadian          | Tingkat | Status | Tanggal | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran Data    | Critical | open   | 4       | diftya  |
+---+-----+-----+-----+-----+
```

Gambar 4.16 Output Pencarian Insiden Berdasarkan ID

```
=== MENU SEARCHING ===
1. Cari berdasarkan ID
2. Cari berdasarkan Pendata
Pilih menu: 2
Masukkan nama Pendata: vana
Daftar Insiden vana:

+---+-----+-----+-----+-----+
| ID | Kejadian          | Tingkat | Status | Tanggal          | Pendata |
+---+-----+-----+-----+-----+
| 3  | Ditemukan Malware pada PC kantor | High   | open   | 17 Februari 2026 | vana    |
+---+-----+-----+-----+-----+
| 4  | Percobaan Hijacking          | High   | open   | 15 April 2026    | vana    |
+---+-----+-----+-----+-----+
```

Gambar 4.17 Output Pencarian Insiden Berdasarkan Pendata

5. Langkah-langkah GIT

5.1 GIT Add

```
~/Documents/praktikum-apl/posttest main*
• > git add post-test-apl-7/
```

Gambar 5.1 Git Add

Dilakukan penambahan *project* ke github yaitu dengan git add.

5.2 GIT Commit

```
~/Documents/praktikum-apl/posttest main*
• > git commit -m "Horeeeeeeeeeee posttest terakhirrr"
[main b0cd04c] Horeeeeeeeeeee posttest terakhirrr
2 files changed, 718 insertions(+)
create mode 100755 posttest/post-test-apl-7/2509106042-Diftya_Azzahra-PT-7
create mode 100644 posttest/post-test-apl-7/2509106042-Diftya_Azzahra-PT-7.cpp
```

Gambar 5.2 Git Commit

Dilakukan commit ke github yaitu dengan *message* berupa Horeeeeeeeeeee posttest terakhirrr.

5.3 GIT Push

```
~/Documents/praktikum-apl/posttest main* ↑
• > git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 108.23 KiB | 6.76 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/HikaruYui/praktikum-apl.git
 63528ea..9da33fd main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3 Git Push

Posttest pun berhasil terpush ke github.